

Identity and Access Management: Lab Guide

Thirty-two hands-on labs, one per chapter, built on a single lab environment you stand up in Lab 03 and extend to the end of the course.

How to use this guide

Each lab maps to one video lesson and produces that chapter's key artifact. Labs are cumulative: the directory, identity provider, and personas you build early are the same ones you govern, attack, federate, and troubleshoot later. Complete them in order.

Every lab follows the same shape: an **objective**, numbered **tasks**, a **done when** checklist, and the **deliverable** you keep in your lab notebook. Hands-on labs also include a **break it** exercise: a deliberate fault to cause, observe, and fix. Take a snapshot before every break-it exercise.

Keep a lab notebook (a Git repository works well) with one folder per lab. Deliverables from early labs are inputs to later ones.

The lab environment

Everything is open source and runs on one machine with Docker Compose. Minimum: 4 CPU cores, 16 GB RAM, 30 GB free disk, and Docker Desktop or Docker Engine with the Compose plugin. A second Keycloak realm named `c1oud` stands in for a cloud identity tenant in the hybrid and federation labs.

Component	Tool	Address
Directory	OpenLDAP	<code>ldap://localhost:389</code>
Identity provider	Keycloak (realm <code>halden</code>)	<code>http://keycloak:8080</code>
App 1: Dashboards	Grafana, native OIDC	<code>http://localhost:3000</code>
App 2: Intranet	whoami behind oauth2-proxy	<code>http://localhost:4180</code>
Policy engine	Open Policy Agent (Lab 14)	<code>http://localhost:8181</code>
Secrets vault	Vault or OpenBao, dev mode (Lab 22)	<code>http://localhost:8200</code>
Simulated cloud tenant	Keycloak realms <code>c1oud</code> , <code>partner</code> , <code>shop</code>	<code>http://keycloak:8080</code>

Workstation tools: `ldap-utils` (`ldapsearch`, `ldapadd`, `ldapmodify`), `curl`, `jq`, `oathtool`, `step CLI`, Python 3 with `bcrypt`, `argon2-cffi`, `networkx`, `ldap3`, and a browser with developer tools and a second profile for private sessions.

The organization and its personas

Halden & Co. is a fictional 40-person distribution company with Finance, Sales, IT, and Operations departments. Base DN: `dc=halden,dc=lab`. Five personas recur through every lab:

Persona	uid	Role in the labs
Priya Shah, employee	<code>pshah</code>	Finance analyst; later moves to Sales (the mover)
Marco Ruiz, contractor	<code>mruiz</code>	Six-month IT project; fixed end date, sponsor-owned
Dana Okafor, admin	<code>dokafor</code> / <code>adm-dokafor</code>	IT administrator; holds a separate privileged account
Reporting service	<code>svc-reporting</code>	Non-human identity that reads Finance data nightly
Lee Tran, leaver	<code>ltran</code>	Sales rep who resigns during the course

Ground rules

- Never practice on production identities, a work tenant, or real people's data. Use only the lab and a dedicated cloud tenant.
- Lab passwords are throwaway. Keep them in a `.env` file that is excluded from Git.
- Offensive techniques appear only to test your own defenses, and only inside the lab.
- Take a snapshot before every break-it exercise and restore it afterward. The scripts are in Lab 03.

Conventions

- Grey boxes hold commands or file contents to type. Outlined white boxes show the expected output. Your IDs, timestamps, and hashes will differ.
- Run all commands from the lab folder `~/halden-lab` unless a step says otherwise. Values like `$LDAP_ADMIN_PASSWORD` come from the `.env` file; run `set -a; source .env; set +a` in each new terminal.
- Keycloak steps name the admin console path, e.g. *Clients* → *grafana* → *Credentials*. Always check the realm selector (top left) first.
- Every lab ends with a checklist. Do not start the next lab until every box is ticked.
- Solutions to every break-it exercise and design task are in the Answer Key at the end of the guide.

Foundations

Vocabulary, the identity data model, the lab itself, the directory, and the threat lens.

LAB 01

What Identity and Access Management Actually Is

Prerequisites: none · Artifact: one-page IAM concept map

Objective. Build the concept map you will return to at each part boundary. You will place the four questions, the four A's, and the IAM disciplines on one page, then pin one real login to it.

TASKS

1. Create the lab notebook. Everything you produce in this course lives here.

```
mkdir -p ~/halden-lab/notebook/lab01 && cd ~/halden-lab
git init
printf '.env\nsnapshots/\n' > .gitignore
```

Initialized empty Git repository in /home/you/halden-lab/.git/

2. On one page (paper, a whiteboard photo, or a diagram tool), draw four boxes in a row: **Authentication**, **Authorization**, **Administration**, **Audit**. Under each, write the question it answers: *who are you and how do you prove it, what may you do, who set it up and keeps it correct, who decided, and can we show it*.
3. Below the four boxes, add the seven disciplines: directory services, authentication, federation, IGA, PAM, CIAM, non-human identity. Draw an arrow from each discipline to the A (or A's) it mainly serves.
4. Trace a login you did today, such as signing into your email. Write each step on a sticky note and place it on the map: typing the username, the password check, the MFA prompt, the inbox loading, the login appearing in your account's activity page.
5. Add the five stakeholder groups (security, IT operations, the business, compliance, users) around the edge. Next to each, write the one thing it wants from IAM in five words or fewer.
6. Save a photo or export as notebook/lab01/concept-map.png and commit it.

```
git add notebook && git commit -m "Lab 01: IAM concept map"
```

DONE WHEN

- All four A's appear with their question, and none overlap.
- Every discipline has at least one arrow.
- Your traced login has at least one step under Authentication, Authorization, and Audit.

- The map is committed to the notebook.

Break it

Classify each statement under one A: “The password was wrong.” “She can see the payroll folder but shouldn't.” “Nobody knows who approved his admin rights.” “The account was created with the wrong manager.” If one fits two boxes, write down why. The distinction matters in Lab 31.

Reset: none needed. You will revise this map at the end of Parts II, III, IV, and VII.

LAB 02

Identity, Accounts, and Attributes

Prerequisites: Lab 01; Python 3 · Artifact: identity data model with authoritative owners

Objective. Work with three messy source exports, match each person across them with no shared key, and record which system owns each attribute.

TASKS

1. Create three source files in notebook/lab02/. They are deliberately inconsistent.

```
# hr.csv – the HR system
employee_id,first,last,dept,title,status,end_date
1001,Priya,Shah,Finance,Financial Analyst,active,
1002,Marco,Ruiz,IT,Contractor - Migration,active,2027-03-31
1003,Dana,Okafor,IT,Systems Administrator,active,
1004,Lee,Tran,Sales,Account Executive,terminated,2026-09-01

# mail.csv – the email system
address,display_name
priya.shah@halden.lab,Priya Shah
p.shah@halden.lab,P Shah
mruiz@halden.lab,Marco Ruiz
dana.okafor@halden.lab,Dana Okafor
lee.tran@halden.lab,Lee Tran

# crm.csv – a SaaS app export
login,full_name,department
pshah,Priya Shah,Finance
ltran,Lee Tran,Sales
jbloggs,Joe Bloggs,Sales
```

2. Save this correlation script as `correlate.py`. It builds a match key from first initial plus surname, the same rule a basic identity tool would start with.

```

import csv, re
def key(first, last): return (first[0] + last).lower()
def split(name):
    parts = name.split(); return parts[0], parts[-1]
hr = {key(r['first'], r['last']): r for r in csv.DictReader(open('hr.csv'))}
matches = {k: { 'hr': v['employee_id'], 'mail': [], 'crm': []} for k, v in hr.items()}
orphans = []
for r in csv.DictReader(open('mail.csv')):
    k = key(*split(r['display_name']))
    (matches[k]['mail'].append(r['address'])) if k in matches else
orphans.append(('mail', r['address']))
for r in csv.DictReader(open('crm.csv')):
    k = key(*split(r['full_name']))
    (matches[k]['crm'].append(r['login'])) if k in matches else
orphans.append(('crm', r['login']))
for k, m in matches.items():
    status = hr[k]['status']
    print(f"{m['hr']} {k:8} {status:10} mail={m['mail']} crm={m['crm']}")
print('ORPHANS:', orphans)

```

```
cd notebook/lab02 && python3 correlate.py
```

```

1001 pshah      active      mail=['priya.shah@halden.lab', 'p.shah@halden.lab'] crm=['pshah']
1002 mruiz      active      mail=['mruiz@halden.lab'] crm=[]
1003 dokafor    active      mail=['dana.okafor@halden.lab'] crm=[]
1004 ltran      terminated mail=['lee.tran@halden.lab'] crm=['ltran']
ORPHANS: [('crm', 'jbloggs')]

```

3. Read the output and list every problem in findings.md. Look for a duplicate identity (two mailboxes for one person), an orphaned account (an app login with no HR record), and a leaver who still has active accounts.
4. Write the data model in data-model.md as a table with the columns *attribute · meaning · authoritative source · consumers*. Include at least: employee ID, legal name, display name, department, title, manager, employment status, end date, email address, and username. Choose exactly one owner for each attribute.
5. Choose the stable unique identifier that will join these systems from now on. Write one paragraph explaining why names and email addresses are poor choices.

DONE WHEN

- Your script output matches the expected output above.
- findings.md lists at least one duplicate, one orphan, and one leaver with live access.
- Every attribute in data-model.md has exactly one authoritative source.
- You have chosen and justified a stable identifier.

Break it

Add an employee 1005,Paul,Shah,Sales,... to `hr.csv` and run the script again. What happens to Priya's accounts? Then change the match key so the collision can't happen, using only data the sources actually contain.

Reset: remove the 1005 line. Keep these CSVs: Lab 18 uses `hr.csv` as the authoritative source.

LAB 03

Building Your IAM Lab

Prerequisites: Docker with Compose, ldap-utils, curl, jq · Artifact: directory + IdP + two apps, verified end to end

Objective. Stand up the environment every later lab uses. You will run OpenLDAP, Keycloak, Grafana, and an intranet app behind oauth2-proxy. At the end, a directory user can sign into both apps with single sign-on. You will also create the snapshot and restore scripts.

TASKS

1. Make the hostname keycloak resolve on your machine. This lets your browser and the containers use the same Keycloak address, which keeps token issuers consistent.

```
echo "127.0.0.1 keycloak" | sudo tee -a /etc/hosts      # Windows:
C:\Windows\System32\drivers\etc\hosts
```

2. Create .env with lab-only secrets. The client secrets stay blank until step 8.

```
cat > .env <<EOF
LDAP_ADMIN_PASSWORD=Lab-LdapAdmin-01
KC_ADMIN_PASSWORD=Lab-KcAdmin-01
USER_PASSWORD=Lab-Pass-2026!
GRAFANA_CLIENT_SECRET=
INTRANET_CLIENT_SECRET=
COOKIE_SECRET=$(openssl rand -base64 32 | tr -s '+/' '-_')
EOF
```

3. Create compose.yaml:

```

name: halden
services:
  openldap:
    image: osixia/openldap:1.5.0
    environment:
      LDAP_ORGANISATION: "Halden and Co"
      LDAP_DOMAIN: halden.lab
      LDAP_ADMIN_PASSWORD: ${LDAP_ADMIN_PASSWORD}
    ports: ["389:389"]
    volumes: [ldap-data:/var/lib/ldap, ldap-config:/etc/ldap/slapd.d]
  keycloak:
    image: quay.io/keycloak/keycloak:26.0
    command: start-dev
    environment:
      KC_BOOTSTRAP_ADMIN_USERNAME: admin
      KC_BOOTSTRAP_ADMIN_PASSWORD: ${KC_ADMIN_PASSWORD}
    ports: ["8080:8080"]
    volumes: [kc-data:/opt/keycloak/data]
  grafana:
    image: grafana/grafana-oss:11.2.0
    ports: ["3000:3000"]
    environment:
      GF_SERVER_ROOT_URL: http://localhost:3000
      GF_AUTH_GENERIC_OAUTH_ENABLED: "true"
      GF_AUTH_GENERIC_OAUTH_NAME: Halden SSO
      GF_AUTH_GENERIC_OAUTH_CLIENT_ID: grafana
      GF_AUTH_GENERIC_OAUTH_CLIENT_SECRET: ${GRAFANA_CLIENT_SECRET}
      GF_AUTH_GENERIC_OAUTH_SCOPES: openid profile email
      GF_AUTH_GENERIC_OAUTH_AUTH_URL: http://keycloak:8080/realms/halden/protocol/openid-
connect/auth
      GF_AUTH_GENERIC_OAUTH_TOKEN_URL: http://keycloak:8080/realms/halden/protocol/openid-
connect/token
      GF_AUTH_GENERIC_OAUTH_API_URL: http://keycloak:8080/realms/halden/protocol/openid-
connect/userinfo
      GF_AUTH_GENERIC_OAUTH_ALLOW_SIGN_UP: "true"
    volumes: [grafana-data:/var/lib/grafana]
  whoami:
    image: traefik/whoami
  oauth2-proxy:
    image: quay.io/oauth2-proxy/oauth2-proxy:v7.6.0
    ports: ["4180:4180"]
    command:
      - --provider=keycloak-oidc
      - --oidc-issuer-url=http://keycloak:8080/realms/halden
      - --client-id=intranet
      - --client-secret=${INTRANET_CLIENT_SECRET}
      - --cookie-secret=${COOKIE_SECRET}
      - --cookie-secure=false
      - --email-domain=*
      - --insecure-oidc-allow-unverified-email
      - --code-challenge-method=S256
      - --upstream=http://whoami:80
      - --http-address=0.0.0.0:4180

```

```
- --redirect-url=http://localhost:4180/oauth2/callback
restart: on-failure
volumes: {ldap-data: {}, ldap-config: {}, kc-data: {}, grafana-data: {}}
```

4. Start the directory and the IdP first, and wait until Keycloak reports that it is running.

```
set -a; source .env; set +a
docker compose up -d openldap keycloak
docker compose logs -f keycloak | grep -m1 "Running the server"
```

```
keycloak-1 | ... INFO [io.quarkus] Running the server in development mode. DO NOT use this
configuration in production.
```

5. Load the base tree and the five personas. Save as `ldif/base.ldif`:

```
dn: ou=people,dc=halden,dc=lab
objectClass: organizationalUnit
ou: people

dn: ou=groups,dc=halden,dc=lab
objectClass: organizationalUnit
ou: groups

dn: ou=service,dc=halden,dc=lab
objectClass: organizationalUnit
ou: service

dn: uid=pshah,ou=people,dc=halden,dc=lab
objectClass: inetOrgPerson
uid: pshah
cn: Priya Shah
givenName: Priya
sn: Shah
mail: pshah@halden.lab
employeeNumber: 1001
departmentNumber: Finance
title: Financial Analyst
employeeType: employee

# Repeat the pshah block for:
#   mruiiz  Marco Ruiz    1002 IT      Contractor - Migration  employeeType: contractor
#   dokafor Dana Okafor   1003 IT      Systems Administrator   employeeType: employee
#   ltran   Lee Tran      1004 Sales   Account Executive       employeeType: employee

dn: uid=svc-reporting,ou=service,dc=halden,dc=lab
objectClass: inetOrgPerson
uid: svc-reporting
cn: Reporting Service
sn: Service
employeeType: service
description: Owner: Finance Systems (pshah)
```

```

ldapadd -x -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w "$LDAP_ADMIN_PASSWORD" -
f ldif/base.ldif
for u in pshah mruiz dokafor ltran; do
    ldappasswd -x -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w "$LDAP_ADMIN_PASSWORD" \
        -s "$USER_PASSWORD" uid=$u,ou=people,dc=halden,dc=lab
done

```

```

adding new entry "ou=people,dc=halden,dc=lab"
adding new entry "ou=groups,dc=halden,dc=lab"
...
adding new entry "uid=svc-reporting,ou=service,dc=halden,dc=lab"

```

6. Test that a user can bind (authenticate) to the directory with their own password. `ldapwhoami` returns the DN of whoever authenticated.

```

ldapwhoami -x -H ldap://localhost -D uid=pshah,ou=people,dc=halden,dc=lab -w "$USER_PASSWORD"

```

```

dn:uid=pshah,ou=people,dc=halden,dc=lab

```

7. Connect Keycloak to the directory. Open `http://keycloak:8080` and sign in as admin. Create a realm named `halden`. Then go to *User federation* → *Add LDAP providers* and set:
- Vendor *Other* · Connection URL `ldap://openldap:389` · Bind DN `cn=admin,dc=halden,dc=lab` and its password. *Lab 05 flags this admin bind as a finding.*
 - Edit mode *READ_ONLY* · Users DN `ou=people,dc=halden,dc=lab` · Username, RDN, and UUID attributes `uid, uid, entryUUID` · User object classes `inetOrgPerson` · Trust email *On*
- Save, then choose *Action* → *Sync all users*.

```

Sync of users finished successfully. 4 imported users, 0 updated users

```

8. Register the two apps. In *Clients* → *Create client*, create an OpenID Connect client named `grafana`. Turn on client authentication, keep the standard flow, and set the redirect URI to `http://localhost:3000/login/generic_oauth`. Create a second client named `intranet` with redirect URI `http://localhost:4180/oauth2/callback`. On the intranet client, open *Client scopes* → *intranet-dedicated* → *Add mapper* → *By configuration* → *Audience*, and set Included Client Audience to `intranet`. Copy each client's secret from its *Credentials* tab into `.env`.
9. Start the apps, then check that the IdP publishes its configuration.

```

set -a; source .env; set +a
docker compose up -d
curl -s http://keycloak:8080/realms/halden/.well-known/openid-configuration | jq -r .issuer
docker compose ps --format '{{.Service}}' '{{.State}}'

```

```
http://keycloak:8080/realms/halden
grafana running
keycloak running
oauth2-proxy running
openldap running
whoami running
```

10. Verify end to end in a private browser window. Open `http://localhost:3000`, choose *Sign in with Halden SSO*, and sign in as pshah. Grafana should open with Priya logged in. Now open `http://localhost:4180` in the same window. You should see the whoami page **without a second login prompt**. That is single sign-on working.

```
Hostname: 3f2a...
X-Forwarded-Email: pshah@halden.lab
X-Forwarded-User: 6b1c...
```

11. Create the snapshot and restore scripts. Save them as `snapshot.sh` and `restore.sh`, then run `chmod +x *.sh`.

```
# snapshot.sh NAME
set -e; mkdir -p snapshots; docker compose stop
for v in ldap-data ldap-config kc-data grafana-data; do
    docker run --rm -v halden_${v}:/v -v "$PWD/snapshots:/s" alpine tar czf /s/$1-${v}.tgz -C /v .
done
docker compose start

# restore.sh NAME
set -e; docker compose stop
for v in ldap-data ldap-config kc-data grafana-data; do
    docker run --rm -v halden_${v}:/v -v "$PWD/snapshots:/s" alpine sh -c "rm -rf /v/* /v/.[*];
tar xzf /s/$1-${v}.tgz -C /v"
done
docker compose start
```

```
./snapshot.sh lab03-baseline && ls snapshots
```

```
lab03-baseline-grafana-data.tgz  lab03-baseline-kc-data.tgz
lab03-baseline-ldap-config.tgz  lab03-baseline-ldap-data.tgz
```

DONE WHEN

- ☐ All five containers are running.
- ☐ `ldapwhoami` works for pshah.
- ☐ Keycloak lists the four people from LDAP under *Users*.
- ☐ pshah signs into Grafana, then reaches the intranet with no second prompt.
- ☐ A `lab03-baseline` snapshot exists, and restoring it brings the lab back.

Break it

Take a snapshot named `pre-break`. Change the Grafana redirect URI in Keycloak to `http://localhost:3000/login/wrong` and try to sign in. Note the exact error text and the URL where it appears. Then run `./restore.sh pre-break` and confirm that login works again.

Reset: `./restore.sh lab03-baseline` gets you back here from any later lab. Take a new snapshot at the end of each lab: `./snapshot.sh labNN-end`.

LAB 04

Directory Services, the Foundation Underneath IAM

Prerequisites: Lab 03 · Artifact: a populated directory with organizational and group structure

Objective. Grow the directory to 40 people and give it a sensible group structure. Along the way you will learn the LDAP operations by hand and practice diagnosing the three most common directory errors.

TASKS

1. Generate 35 more employees with this script (gen_users.py). It writes LDIF with realistic departments, titles, and managers.

```
import random; random.seed(7)
first = "Ana Ben Chloe Dev Eli Fay Gus Hana Ivan Jo Kai Lina Max Nia Omar Pia Raj Sara Tom Uma
Vik Wen Yara Zed Ada Bo Cy Di Ed Flo Gil Ida Jem Kit Lou".split()
last = "Ng Park Diaz Cole Moss Hart Lund Rao Sato Kerr".split()
depts = {"Finance": "Accountant", "Sales": "Sales Rep", "IT": "Support Engineer",
"Operations": "Warehouse Lead"}
for i, f in enumerate(first):
    l = random.choice(last); d = random.choice(list(depts)); uid = (f[0] + l).lower() + str(i)
    print(f"dn: uid={uid},ou=people,dc=halden,dc=lab\nobjectClass:
inetOrgPerson\nuid: {uid}\n"
        f"cn: {f} {l}\ngivenName: {f}\nsn: {l}\nmail: {uid}@halden.lab\nemployeeNumber:
{1100+i}\n"
        f"departmentNumber: {d}\ntitle: {depts[d]}\nemployeeType: employee\n")
```

```
python3 gen_users.py > ldif/people.ldif
ldapadd -x -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w "$LDAP_ADMIN_PASSWORD" -f
ldif/people.ldif | tail -1
ldapsearch -x -LLL -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w
"$LDAP_ADMIN_PASSWORD" \
-b ou=people,dc=halden,dc=lab "(objectClass=inetOrgPerson)" dn | grep -c ^dn
```

```
adding new entry "uid=1...34,ou=people,dc=halden,dc=lab"    (surname varies)
39
```

Thirty-nine people plus one service account makes forty identities.

2. Read one entry by hand and name each part of what you see: the DN, the object classes, and the attributes.

```
ldapsearch -x -LLL -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w
"$LDAP_ADMIN_PASSWORD" \
-b dc=halden,dc=lab "(uid=pshah)" + '*'
```

```
dn: uid=pshah,ou=people,dc=halden,dc=lab
objectClass: inetOrgPerson
uid: pshah
cn: Priya Shah
...
entryUUID: 5d0c...      ← operational attribute, returned only when you ask for "+"
createTimestamp: 2026...
```

3. Give Priya a manager, using `ldapmodify`. The manager attribute holds a DN, which is how directories link entries to each other.

```
ldapmodify -x -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w "$LDAP_ADMIN_PASSWORD" <<EOF
dn: uid=pshah,ou=people,dc=halden,dc=lab
changetype: modify
add: manager
manager: uid=dokafor,ou=people,dc=halden,dc=lab
EOF
```

```
modifying entry "uid=pshah,ou=people,dc=halden,dc=lab"
```

4. Create groups in two layers. **Department groups** say who someone is: `grp-finance`, `grp-sales`, `grp-it`, `grp-operations`, `grp-all-staff`. **Access groups** say what someone gets: `app-grafana-viewer`, `app-grafana-editor`, `app-intranet-users`, `grp-lab-admins`. This script writes a `groupOfNames` entry for each department from the `departmentNumber` attribute:

```
B="-x -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w $LDAP_ADMIN_PASSWORD"
for d in Finance Sales IT Operations; do
  g=grp-$(echo $d | tr A-Z a-z)
  { echo "dn: cn=$g,ou=groups,dc=halden,dc=lab"; echo "objectClass: groupOfNames"; echo
    "cn: $g"
    ldapsearch $B -LLL -b ou=people,dc=halden,dc=lab "(departmentNumber=$d)" dn | sed -n
    's/^dn: /member: /p'
  } | ldapadd $B
done
```

Create the access groups yourself in `ldif/access-groups.ldif`. Put `dokafor` in `grp-lab-admins`, the whole Finance department in `app-grafana-viewer`, and every person in `app-intranet-users`.

5. In Keycloak, open *User federation* → *ldap* → *Mappers* → *Add mapper*. Choose type *group-ldap-mapper*, set LDAP Groups DN to `ou=groups,dc=halden,dc=lab`, mode *READ_ONLY*, and membership attribute `member`. Save, then choose *Sync LDAP groups to Keycloak*. Priya's *Groups* tab should now show `grp-finance`, `app-grafana-viewer`, and `app-intranet-users`.
6. Trigger the three classic errors one at a time and write each error code into `notebook/lab04/errors.md`:

```
ldapwhoami -x -H ldap://localhost -D uid=pshah,ou=people,dc=halden,dc=lab -w wrong      #
failed bind
ldapsearch -x -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w "$LDAP_ADMIN_PASSWORD" -b
ou=staff,dc=halden,dc=lab    # wrong base
ldapsearch -x -LLL -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w "$LDAP_ADMIN_PASSWORD"
-b dc=halden,dc=lab "(uid=pshah)" telephoneNumber    # missing attribute
```

```
ldap_bind: Invalid credentials (49)
...result: 32 No such object
dn: uid=pshah,ou=people,dc=halden,dc=lab          ← entry found, attribute silently absent
```

DONE WHEN

- The directory holds 39 people and one service account.
- Four department groups and four access groups exist, and each has members.
- Keycloak shows LDAP group membership for Priya.
- `errors.md` explains errors 49 and 32 and the silent missing-attribute case.
- You have written one paragraph on what would stop working if the directory went down, and how replication would prevent it.

Break it

Take a snapshot named `pre-break`, then run `docker compose stop openldap`. Try to sign into Grafana as `pshah` in a new private window. Then try as the Keycloak admin in the master realm. Explain why one login works and the other fails. Start the directory again with `docker compose start openldap`.

Reset: `./snapshot.sh lab04-end`. Later labs expect this population.

LAB 05

The IAM Security Mindset

Prerequisites: Lab 04 · Artifact: identity threat model applied to the lab

Objective. Look at the lab the way an attacker would. List what can be stolen, estimate how far each stolen identity could reach, and record findings that later labs will fix.

TASKS

1. Create notebook/lab05/threat-model.md with this template:

```
| Asset / identity | Attack surface (credential, session, token, trust, admin plane) |  
| Kill-chain stage it enables | Blast radius if compromised | Current control | Gap | Fixed in  
lab |
```

2. Check whether the directory answers anonymous queries. Run the search below with no credentials at all.

```
ldapsearch -x -LLL -H ldap://localhost -b dc=halden,dc=lab "(uid=*)" uid | head
```

If entries come back, anyone on the network can list every username, which is the raw material for password spraying. Record it as a finding. If you get `No such object (32)` or no results, record that anonymous read is off.

3. Find where privileged credentials live. Search your lab files for secrets:

```
grep -rE "PASSWORD|SECRET" --include='*.yaml' --include='.env' .
```

```
./.env:1:LDAP_ADMIN_PASSWORD=Lab-LdapAdmin-01  
./.env:2:KC_ADMIN_PASSWORD=Lab-KcAdmin-01  
...
```

Now look at the Keycloak LDAP connection from Lab 03. It binds as `cn=admin`, the account that can rewrite the whole directory. Add a row for it.

4. Fill in one row for each persona and each of these: the Keycloak master admin, the LDAP admin, and the Grafana and intranet client secrets. For blast radius, answer one question: *if this identity is compromised, what can it reach?* Use the groups from Lab 04 to answer it.
5. Walk one scenario through the kill chain in `kill-chain.md`. Lee Tran has left, but his password still works. Write one line for each stage: initial access, escalation, lateral movement, persistence. Name the lab control, existing or missing, that would stop each one.
6. Rank the top five gaps. For each, write the least-privilege or zero-trust principle that fixes it and the lab where you will fix it.

DONE WHEN

- The threat model covers all 5 personas and at least 4 system identities.
- The admin bind and the anonymous-read result are recorded.
- The kill chain names a control for every stage.
- Each of the top five gaps points to a later lab.

Break it

Prove the leaver scenario is real. In the directory, Lee is still an ordinary active account. Run `ldapwhoami` as `ltran`, then sign into the intranet as him. Both work. Record the evidence. Do not fix it yet: Lab 17 does that properly.

Reset: nothing changed. Revisit this threat model at the end of every part and mark gaps as closed.

Authentication

From the first password check to passkeys, device certificates, and the session that follows the login.

LAB 06

How Authentication Works

Prerequisites: Lab 04 snapshot; browser developer tools · Artifact: annotated authentication flow diagram

Objective. Watch a real login step by step, find where the claim, the credential check, and the session creation each happen, and compare local directory authentication with centralized authentication.

TASKS

1. Turn on login auditing. In the halden realm, go to *Realm settings* → *Events* → *User events settings*, switch *Save events* on, and save.
2. Open a private window with developer tools on the *Network* tab and *Preserve log* ticked. Sign into Grafana as pshah. Find these four requests and copy their URLs into notebook/lab06/trace.md:

```
1 GET keycloak:8080/realms/halden/protocol/openid-connect/auth?
  client_id=grafana&response_type=code&... ← the claim begins
2 POST keycloak:8080/realms/halden/login-actions/authenticate?...
  ← credential submitted
3 302 localhost:3000/login/generic_oauth?code=...&state=...
  ← proof returned to app
4 GET localhost:3000/ (Set-Cookie: grafana_session=...)
  ← app session created
```

The token exchange, where Grafana swaps the code for tokens, happens server to server. It does not appear in your browser. Write down why that is a security benefit.

3. Check the audit trail at *Events* → *User events*. You should see a LOGIN event for pshah with client grafana and your IP address, and a CODE_TO_TOKEN event for the exchange.
4. Compare with local authentication. The command below makes the app check the password against the directory itself, with no IdP involved:

```
ldapwhoami -x -H ldap://localhost -D uid=pshah,ou=people,dc=halden,dc=lab -w "$USER_PASSWORD"
```

In trace.md, list three things the local bind cannot do that the IdP flow did: SSO, MFA, and a central audit event. Also note where the password traveled in each case.

5. Look at the assurance claim. Go to *Clients* → *grafana* → *Client scopes* → *Evaluate*, choose user pshah, and open *Generated ID token*. Find the acr claim.

```
"acr": "1",  
"preferred_username": "pshah",
```

acr records how strongly the user was authenticated. Lab 08 raises it.

6. Draw the flow diagram as `auth-flow.png`. Use lanes for browser, Grafana, Keycloak, and OpenLDAP. Mark every check: user lookup, password bind, redirect-URI match, code exchange with client secret, and session cookie issue.

DONE WHEN

- `trace.md` contains all four requests.
- The LOGIN and CODE_TO_TOKEN events are visible.
- The diagram marks at least five checks and labels identification, authentication, and session.

Break it

Sign in as pshah with a wrong password, then as nobody. Compare the two LOGIN_ERROR events. Is the error the same? Does the login page show the user the same message both times? Explain why the page should not reveal which usernames exist.

Reset: keep events on; later labs read them. `./snapshot.sh lab06-end`.

LAB 07

Passwords and Their Discontents

Prerequisites: Lab 06; `pip install bcrypt argon2-cffi` · Artifact: defensible password policy

Objective. See for yourself why passwords must be stored with slow, salted hashes. Then configure a policy based on current guidance and test lockout.

TASKS

1. Measure hash speed. Save as `hash_speed.py` and run it:

```
import hashlib, time, bcrypt
from argon2 import PasswordHasher
pw = b"Summer2026!"; ph = PasswordHasher()
def ms(f, n):
    s = time.perf_counter()
    for _ in range(n): f()
    return (time.perf_counter() - s) / n * 1000
print(f"md5          {ms(lambda: hashlib.md5(pw).digest(), 100000):.5f} ms")
print(f"sha256       {ms(lambda: hashlib.sha256(pw).digest(), 100000):.5f} ms")
print(f"bcrypt 12    {ms(lambda: bcrypt.hashpw(pw, bcrypt.gensalt(12)), 5):.1f} ms")
print(f"argon2id     {ms(lambda: ph.hash(pw), 5):.1f} ms")
```

```
md5          0.00030 ms
sha256       0.00035 ms
bcrypt 12    210.4 ms
argon2id     45.8 ms
```

Work out how many guesses per second an attacker gets against each: $1000 \div$ the milliseconds. Record the results.

2. See what salt does. Hash the same password twice with each method:

```
python3 -c "import
hashlib;print(hashlib.sha256(b'Summer2026!').hexdigest());print(hashlib.sha256(b'Summer2026!')
.hexdigest())"
python3 -c "import
bcrypt;print(bcrypt.hashpw(b'Summer2026!',bcrypt.gensalt()));print(bcrypt.hashpw(b'Summer2026!
',bcrypt.gensalt()))"
```

```
7c1e...a4    ← identical: one lookup table cracks every user with this password
7c1e...a4
b'$2b$12$Q9x...' ← different every time: each user must be attacked separately
b'$2b$12$Lm3...'
```

3. Check how the directory stores Priya's password:

```
ldapsearch -x -LLL -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w
"$LDAP_ADMIN_PASSWORD" \
  -b dc=halden,dc=lab "(uid=pshah)" userPassword | sed -n 's/^userPassword:: //p' | base64 -
d; echo
```

```
{SSHA}kq3v...
```

SSHA is salted, but it is a fast hash. Record this as a finding. Then write one sentence on why encrypting passwords would be worse: whoever holds the key can recover every password.

4. Configure the policy in Keycloak. LDAP users in read-only mode keep their directory password, so create a local test user first: *Users* → *Add user*, username `local-test`. Then go to *Authentication* → *Policies* → *Password policy* and add: *Minimum length 12, Not username, Not email, Not recently used 5*. Do not add rules for uppercase, digits, or special characters, and do not add expiry.
5. Add breached-password screening. Keycloak reads a blacklist file from its data folder:

```
printf 'summer2026!\npassword1234\nhalden123456\nwelcome12345\n' > blacklist.txt
docker compose exec keycloak mkdir -p /opt/keycloak/data/password-blacklists
docker compose cp blacklist.txt keycloak:/opt/keycloak/data/password-blacklists/blacklist.txt
```

Add the policy *Password blacklist* with the value `blacklist.txt`. Then set passwords for `local-test` under *Users* → *local-test* → *Credentials*:

```
Summer2026!           → rejected: blacklisted
short1                → rejected: minimum length
correct horse battery lab → accepted
```

6. Turn on lockout at *Realm settings* → *Security defenses* → *Brute force detection*. Choose *Lockout temporarily* with max login failures 5. Fail six logins as `local-test`. The sixth shows the generic error, and *Users* → *local-test* shows the account as temporarily locked.
7. Write `password-policy.md`. Give each rule a reason tied to an attack from the video: brute force, spraying, stuffing, or cracking.

DONE WHEN

- The hash timings and guesses per second are recorded.
- Blacklisted and short passwords are rejected, and a long passphrase is accepted.
- Lockout triggers on the sixth failure.
- The policy has no composition rules and no forced rotation, and explains why.

Break it

Password spraying tries one common password against many users, staying under the lockout limit for each one. Try the same wrong password once against five different LDAP users with `ldapwhoami`. Does the directory lock anyone out? Does Keycloak's brute-force detection see these attempts? Explain both answers.

Reset: `unlock local-test` from its user page. `./snapshot.sh lab07-end`.

LAB 08

Multi-Factor Authentication

Prerequisites: Lab 07; oathtool; an authenticator app on your phone (optional) · Artifact: MFA strategy matrix

Objective. Enroll TOTP for a user, compute the codes yourself to see how they work, and write a strategy that matches factor strength to how sensitive each resource is.

TASKS

1. Require enrollment for Dana. Go to *Users* → *dokafor* → *Details* → *Required user actions*, add *Configure OTP*, and save. Keycloak's default browser flow already asks for OTP from any user who has one configured.
2. Sign into Grafana as dokafor. On the OTP setup page, choose *Unable to scan?* and copy the secret, for example JBSW Y3DP EHPK 3PXP. Compute the current code from the command line:

```
SECRET="JBSWY3DPEHPK3PXP"      # your secret, spaces removed
oathtool --totp -b "$SECRET"
```

492039

Enter it to finish enrollment. If you have a phone app, scan the QR code too. It will show the same code.

3. See how TOTP works by computing the code in about ten lines of Python (totp.py):

```
import base64, hmac, hashlib, struct, time, sys
key = base64.b32decode(sys.argv[1])
counter = int(time.time()) // 30          # the time step: same on phone and server
mac = hmac.new(key, struct.pack(">Q", counter), hashlib.sha1).digest()
off = mac[-1] & 0x0F                      # dynamic truncation
code = (struct.unpack(">I", mac[off:off+4])[0] & 0x7FFFFFFF) % 1_000_000
print(f"{code:06d} (valid for {30 - int(time.time()) % 30}s)")
```

```
python3 totp.py "$SECRET"; oathtool --totp -b "$SECRET"
```

771224 (valid for 18s)
771224

4. Sign out and sign in again as dokafor. Confirm the OTP prompt appears after the password. Under *Events* you should see LOGIN with the credential type recorded, and in the ID token (Evaluate tab) the acr is still 1 until you configure level-of-authentication mapping.
5. Stretch: step-up authentication. Under *Realm settings* → *General* → *ACR to LoA mapping*, add password = 1 and mfa = 2. Build a copy of the browser flow that uses *Condition - Level of Authentication* subflows, following the Keycloak Server Admin guide section “Creating a browser login flow with step-up

mechanism". Test it by adding `&acr_values=mfa` to an auth URL.

6. Fill in `mfa-strategy.md`. Rows: intranet, Grafana viewer, Grafana editor, Keycloak admin console, LDAP admin. Columns: sensitivity, required factor, allowed fallback, phishing-resistant (yes/no). Compare OTP app, push, SMS, hardware key, and platform biometric on two points each: what attack defeats it, and how much friction it adds.

DONE WHEN

- Dana cannot sign in without a valid TOTP.
- `totp.py` and `oathtool` produce the same code.
- The strategy requires the strongest factor for admin access and rules out SMS there, with reasons.

Break it

Simulate a device clock that has drifted. Generate a code three minutes in the future and try it:

```
oathtool --totp -b "$SECRET" --now "$(date -u -d '+3 min' '+%Y-%m-%d %H:%M:%S UTC')" #  
macOS: brew install coreutils, use gdate
```

It is rejected. Find the setting in *Authentication* → *Policies* → *OTP policy* that controls how much drift is tolerated. Explain what you would give up by raising it.

Reset: keep Dana's OTP; she is the admin persona. Save her secret in `.env` as `DANA_TOTP`. `./snapshot.sh`
lab08-end.

LAB 09

Passwordless and Modern Authentication

Prerequisites: Lab 08; Chrome or Edge; step CLI; openssl · Artifact: passwordless adoption plan with phased fallback

Objective. Prove possession of a private key by hand, register a passkey on a virtual authenticator, and roll passwordless out to one app while every other app keeps using passwords.

TASKS

1. Do a challenge-response by hand. This is the core of every passkey and smart card: the server sends a random challenge, and only the private key can sign it.

```
mkdir -p notebook/lab09 && cd notebook/lab09
openssl genpkey -algorithm ec -pkeyopt ec_paramgen_curve:P-256 -out priya.key      # stays on
the "device"
openssl pkey -in priya.key -pubout -out priya.pub                               #
registered with the server
openssl rand -hex 32 > challenge.txt                                           # server
sends this
openssl dgst -sha256 -sign priya.key -out sig.bin challenge.txt                 #
device signs
openssl dgst -sha256 -verify priya.pub -signature sig.bin challenge.txt         #
server verifies
```

Verified OK

Note what the server stores: only the public key. There is no shared secret for an attacker to steal from it.

2. Make keycloak:8080 a secure context. WebAuthn requires HTTPS, and the lab runs over HTTP. Open `chrome://flags/#unsafely-treat-insecure-origin-as-secure`, add `http://keycloak:8080`, and relaunch the browser.
3. Add a virtual authenticator, so you don't need a real security key. Open DevTools, then *More tools* → *WebAuthn*, and tick *Enable virtual authenticator environment*. Add one with protocol *ctap2*, transport *internal*, and resident key and user verification both on. Keep DevTools open for the next steps.
4. Build a passwordless flow. Go to *Authentication* → *Flows*, find *browser*, and choose *Duplicate*, naming it *browser-passkey*. In the copy's *forms* subflow, delete *Username Password Form* and add *Username Form* (Required). Then add a subflow *auth-method* (Required), and inside it add *WebAuthn Passwordless Authenticator* (Alternative) and *Password Form* (Alternative). Under *Required actions*, enable *Webauthn Register Passwordless*.
5. Phase 1 rollout: apply the new flow to the intranet only. Go to *Clients* → *intranet* → *Advanced* → *Authentication flow overrides* and set Browser Flow to *browser-passkey*. Grafana is untouched.

6. Add the required action *Webauthn Register Passwordless* to pshah. Sign into `http://localhost:4180`. After the password step, Keycloak asks you to register a passkey, and the virtual authenticator creates one. The WebAuthn panel now lists a credential with RP ID `keycloak`.
7. Sign out and sign in to the intranet again. Enter pshah, choose the passkey option, and you are in with no password. Then sign into Grafana and confirm it still asks for a password. That is the side-by-side transition working.
8. Write `passwordless-plan.md` in phases: pilot on low-risk apps → admins move to device-bound keys → broad rollout with synced passkeys → passwords removed from high-risk apps. Give each phase an entry criterion, a fallback, and a recovery path for a lost device.

DONE WHEN

- The manual signature check prints `Verified OK`.
- Priya reaches the intranet with a passkey and no password.
- Grafana is unchanged.
- The plan states which authenticators suit which personas: synced versus device-bound.

Break it

Phishing resistance, demonstrated. Add `http://127.0.0.1:8080` to the same Chrome flag, then open `http://127.0.0.1:8080/realms/halden/account`. It is the same server, but under a different origin, the way a lookalike phishing domain would be. Try the passkey. Why does the authenticator refuse, even though the page looks identical?

Reset: virtual authenticators vanish when DevTools closes; Priya's registered credential stays in Keycloak. Remove it under *Users* → *pshah* → *Credentials* if you want a clean start. `./snapshot.sh`
`lab09-end`.

LAB 10

Biometrics and Device Identity

Prerequisites: Lab 09; step CLI · Artifact: device and biometric factor policy

Objective. Tune a biometric threshold using sample match scores, check the user-verification flag in WebAuthn, and give a device its own identity with a certificate it must prove it holds.

TASKS

1. Explore the tradeoff between false accepts and false rejects. Save as `far_frr.py` and run it. The scores are made-up matcher outputs from 0 to 100.

```
genuine = [91, 88, 95, 72, 84, 90, 67, 93, 89, 79]    # the right person
impostor = [22, 41, 35, 58, 12, 66, 30, 47, 71, 28]   # someone else
for t in (50, 60, 70, 80):
    far = sum(s >= t for s in impostor) / len(impostor)
    frr = sum(s < t for s in genuine) / len(genuine)
    print(f"threshold {t}: FAR {far:.0%} FRR {frr:.0%}")
```

```
threshold 50: FAR 30% FRR 0%
threshold 60: FAR 20% FRR 0%
threshold 70: FAR 10% FRR 10%
threshold 80: FAR 0% FRR 30%
```

Choose a threshold for unlocking a phone and a different one for approving a payment. Justify both.

2. Check where biometric matching happens. In the WebAuthn DevTools panel from Lab 09, switch *user verification* off on the virtual authenticator and sign into the intranet with the passkey. It fails if Keycloak's policy requires verification. Look at *Authentication* → *Policies* → *Webauthn Passwordless Policy* → *User verification requirement*. Write down what the server actually receives: a **flag** saying “user verified”, never the fingerprint or face data.
3. Create a device certificate authority and a device identity:

```
mkdir -p notebook/lab10/pki && cd notebook/lab10/pki
step certificate create "Halden Device CA" ca.crt ca.key --profile root-ca --no-password -
-insecure
step certificate create laptop-pshah-01 dev.crt dev.key --profile leaf --ca ca.crt --ca-key
ca.key \
    --no-password --insecure --not-after 2160h
step certificate inspect dev.crt --short
```

```
X.509v3 TLS Certificate (ECDSA P-256) [Serial: 1f3a...]
Subject:      laptop-pshah-01
Issuer:       Halden Device CA
Valid from:   2026-09-24T...
to:          2026-12-23T...
```

4. Make the device prove its identity over mutual TLS. Run a server that only accepts devices signed by your CA:

```
# terminal 1: the "resource"
openssl s_server -accept 8443 -cert dev.crt -key dev.key -CAfile ca.crt -Verify 1 -www
# terminal 2: the device connects with its certificate
openssl s_client -connect localhost:8443 -cert dev.crt -key dev.key -CAfile ca.crt -quiet <<<
"GET / HTTP/1.0"
# terminal 2: an unknown device with no certificate
openssl s_client -connect localhost:8443 -CAfile ca.crt -quiet <<< "GET / HTTP/1.0"
```

```
depth=0 CN = laptop-pshah-01 ... verify return:1    ← server log: device identified
...
alert certificate required                          ← unknown device refused
```

To keep the lab simple, the device's certificate doubles as the server certificate here.

5. Write the device posture record that Lab 16 will feed to the policy engine. Save as notebook/lab10/posture-pshah.json:

```
{"device_id": "laptop-pshah-01", "managed": true, "disk_encrypted": true,
  "os_patch_age_days": 12, "cert_valid": true, "owner": "pshah"}
```

6. Write device-biometric-policy.md. Cover: which biometric modalities are allowed; that matching stays on the device; enrollment and fallback; that biometric data is never collected by a server; device certificate lifetime and revocation; and the posture minimums for each sensitivity tier.

DONE WHEN

- Thresholds are chosen and justified for two use cases.
- The mTLS server accepts the enrolled device and refuses the unknown one.
- The policy says what happens when a biometric can't be changed and a device is lost.

Break it

Issue a second device certificate from a different CA (step `certificate create "Rogue CA" ...`) and connect with it. Then issue a valid certificate with `--not-after 1m`, wait two minutes, and connect again. Record the two different errors.

Reset: no containers changed. Keep pki/; Lab 24 reuses the CA idea for workloads.

LAB 11

Sessions, Tokens, and Single Sign-On

Prerequisites: Lab 08 snapshot; step CLI; a second browser profile · Artifact: session and SSO design with lifetime and re-evaluation policy

Objective. Look inside a token, find the three separate sessions a single login creates, replay a stolen session cookie, and then shorten lifetimes so that revoking access actually takes effect.

TASKS

1. Create a command-line test client. Go to *Clients* → *Create client*, name it *cli-test*, turn on client authentication, and tick only *Direct access grants*. Direct grants send the password straight to the IdP, so use this client for lab inspection only. Put its secret in *.env* as *CLI_SECRET*.
2. Get a token and decode it:

```
TOKEN=$(curl -s http://keycloak:8080/realms/halden/protocol/openid-connect/token \
-d grant_type=password -d client_id=cli-test -d client_secret="$CLI_SECRET" \
-d username=pshah -d password="$USER_PASSWORD" -d scope=openid | jq -r .access_token)
echo "$TOKEN" | step crypto jwt inspect --insecure | jq '.payload | {iss, sub, azp, exp, iat, preferred_username, sid}'
```

```
{ "iss": "http://keycloak:8080/realms/halden", "sub": "6b1c...", "azp": "cli-test",
  "exp": 1790251200, "iat": 1790250900, "preferred_username": "pshah", "sid": "a8e2..." }
```

Work out `exp - iat`. The default access-token lifetime is 300 seconds. The token is self-contained: any app can validate it offline using the realm's public keys at `/protocol/openid-connect/certs`.

3. Find the three sessions. Sign into the intranet as *pshah*, then look at *DevTools* → *Application* → *Cookies*. Record the IdP session cookies (*KEYCLOAK_IDENTITY*, *KEYCLOAK_SESSION* on *keycloak*) and the app session cookie (*_oauth2_proxy* on *localhost*). Then open *Sessions* in the admin console and find Priya's IdP session with its two clients.
4. Replay a stolen session. Copy the value of the *_oauth2_proxy* cookie. In a second browser profile, open `http://localhost:4180`, add a cookie with that same name and value in *DevTools*, and reload. You are now Priya, with no password and no MFA.
5. Revoke and observe. In *Sessions*, sign Priya out. Reload the intranet in both profiles. **Both still work.** The app's cookie is valid for 7 days by default and never checks back with the IdP.
6. Add re-evaluation. Add these flags to *oauth2-proxy* in *compose.yaml*, then run `docker compose up -d oauth2-proxy`:

```
- --cookie-refresh=1m
- --cookie-expire=8h
```

In *Realm settings* → *Sessions*, set SSO Session Idle to 30 minutes and SSO Session Max to 10 hours. Repeat tasks 4 and 5. This time, within about a minute of the revocation, the refresh fails and both browsers are sent back to login.

7. Write `session-design.md`. For each app, give the access-token lifetime, app session lifetime, idle timeout, maximum lifetime, re-evaluation interval, and what triggers a forced re-authentication (role change, risk signal, admin revoke).

DONE WHEN

- You decoded a token and explained `iss`, `sub`, `aud/azp`, and `exp`.
- You can name the IdP session and the app session separately.
- Revocation reaches the intranet within about one minute.
- The design justifies each lifetime with a sentence about risk and a sentence about user friction.

Break it

Remove `--cookie-refresh` again and set Keycloak's Access Token Lifespan to 1 minute. Revoke Priya's session. Does the intranet notice? Explain why a short token lifetime at the IdP does nothing when the app never uses the token after login.

Reset: keep `--cookie-refresh=1m`. Delete the cookie you copied into the second profile. `./snapshot.sh lab11-end`.

Authorization

Deciding what an authenticated identity may do: enforcement points, roles, attributes, entitlements, and zero trust.

LAB 12

How Authorization Works

Prerequisites: Lab 11 snapshot · Artifact: authorization architecture diagram separating decision and enforcement

Objective. Up to now, anyone who can log in can use both apps. In this lab you add group claims to the tokens, enforce access at the gateway and inside the app, and make both fail closed.

TASKS

1. Put group membership into the tokens. For each of the grafana and intranet clients, open *Client scopes* → *<client>-dedicated* → *Add mapper* → *By configuration* → *Group Membership*. Set Token Claim Name to groups, turn *Full group path* off, and leave ID token, access token, and userinfo on.
2. Check the claim with the Evaluate tab (*Clients* → *grafana* → *Client scopes* → *Evaluate*, user pshah):

```
"groups": ["grp-finance", "app-grafana-viewer", "app-intranet-users"]
```

3. Enforce at the gateway. oauth2-proxy is a policy enforcement point (PEP) in front of the intranet. Add this flag and run `docker compose up -d oauth2-proxy`:

```
- --allowed-group=app-intranet-users
```

4. Enforce inside the app. Add role mapping to Grafana's environment, then run `docker compose up -d grafana`:

```
GF_AUTH_GENERIC_OAUTH_ROLE_ATTRIBUTE_PATH: "contains(groups[*], 'app-grafana-editor') &&
'Editor' || contains(groups[*], 'app-grafana-viewer') && 'Viewer'"
GF_AUTH_GENERIC_OAUTH_ROLE_ATTRIBUTE_STRICT: "true"
```

ROLE_ATTRIBUTE_STRICT is the fail-closed switch. With it on, a user who matches no rule is refused. Without it, Grafana would let them in as Viewer.

5. Test four cases in private windows and record the result of each:

```
pshah   → Grafana: Viewer      intranet: allowed
dokafor → Grafana: login refused intranet: allowed      (IT isn't in app-grafana-viewer)
mruiz   → Grafana: login refused intranet: allowed
(remove mruiz from app-intranet-users with ldapmodify, then sync LDAP groups in Keycloak)
mruiz   → intranet: 403 Forbidden
```

6. Draw `authz-architecture.png`. Show the PDP, the place where the decision is made (today: group membership in the IdP plus the rules in each app), and each PEP: the gateway, the app, and a future data layer. Mark which PEPs fail closed.
7. Classify the lab's current authorization against the models in the video: DAC, MAC, RBAC, ABAC. Name one place where each one would fit Halden.

DONE WHEN

- ☐ Tokens carry a groups claim.
- ☐ All four test cases match the expected results.
- ☐ The diagram separates decision from enforcement and marks fail-open versus fail-closed.

Break it

The missing check. The `whoami` container has no port published, but it is still reachable from inside the Docker network. Run `docker run --rm --network halden_default nicolaka/netshoot curl -s http://whoami`. It answers with no authentication at all. Explain why enforcement that exists only at the gateway is not enough, and which kind of PEP would close the gap.

Reset: add `mrui` back to `app-intranet-users` and sync. `./snapshot.sh lab12-end`.

LAB 13

Role-Based Access Control

Prerequisites: Lab 12 · Artifact: role model with a separation-of-duties matrix

Objective. Derive roles from existing access, map job-function roles onto access groups, and design separation of duties for a finance process.

TASKS

1. Start from an access export for a fictional finance app, Ledger. Its user list is sample data and doesn't need to match your directory. Save as `notebook/lab13/ledger-access.csv`:

```
user,title,entitlements
pshah,Financial Analyst,view_reports;export_reports
ang0,Accountant,view_reports;create_invoice;create_vendor
bpark1,Accountant,view_reports;create_invoice;create_vendor
cdiaz2,Accountant,view_reports;create_invoice;create_vendor;approve_payment
dcole3,Finance Manager,view_reports;approve_payment;export_reports
emoss4,Finance Manager,view_reports;approve_payment;export_reports
dokafor,Systems Administrator,view_reports;create_vendor;approve_payment;admin_config
```

2. Mine the roles. Group users whose entitlement sets are identical (`role_mining.py`):

```
import csv, collections
groups = collections.defaultdict(list)
for r in csv.DictReader(open('ledger-access.csv')):
    groups[frozenset(r['entitlements'].split(';'))].append(f"{r['user']} ({r['title']})")
for ents, users in sorted(groups.items(), key=lambda g: -len(g[1])):
    print(len(users), sorted(ents), users)
```

```
2 ['create_invoice', 'create_vendor', 'view_reports'] ['ang0 (Accountant)',
'bpark1 (Accountant)']
2 ['approve_payment', 'export_reports', 'view_reports'] ['dcole3 (Finance Manager)', 'emoss4
(Finance Manager)']
1 ['export_reports', 'view_reports'] ['pshah (Financial Analyst)']
1 ['approve_payment', 'create_invoice', 'create_vendor', 'view_reports']
['cdiaz2 (Accountant)']
1 ['admin_config', 'approve_payment', 'create_vendor', 'view_reports'] ['dokafor
(Systems Administrator)']
```

The clusters suggest roles. The single-user rows are exceptions to investigate, not new roles.

3. Write the separation-of-duties matrix. Mark every pair of entitlements that no single person should hold. At minimum: `create_vendor` + `approve_payment` (you could invent a vendor and pay it) and `create_invoice` + `approve_payment`. Find which users above break a rule.
4. Define the role model in `role-model.md`. Base roles on job function, not the org chart:

```
role: Staff (birthright)          → app-intranet-users
role: Finance Analyst inherits Staff → app-grafana-viewer,
ledger:view_reports, ledger:export_reports
role: Accounts Payable inherits Staff → ledger:view_reports,
ledger:create_invoice, ledger:create_vendor
role: Payment Approver inherits Staff → ledger:view_reports, ledger:approve_payment
role: Dashboard Author inherits Finance Analyst → app-grafana-editor
role: Platform Admin   → grp-lab-admins, ledger:admin_config   (privileged; Part V)
SoD: Accounts Payable X Payment Approver
```

5. Implement one role in the lab. Create the LDAP group `role-dashboard-author` containing `pshah`, and add `Priya` to `app-grafana-editor`. This is static assignment: someone must remember to remove it later. Sync groups in Keycloak and sign into Grafana as `Priya`. Her role is now **Editor**.
6. Count the roles you would need if every department-and-location combination got its own role (4 departments × 3 sites × 3 seniority levels). Write two sentences on role explosion.

DONE WHEN

- The mining output matches, and you have explained each exception.
- The SoD matrix flags `cdiaz2` and `dokafor`.
- `Priya` is an Editor in Grafana through a group, not a manual grant.
- You have listed one access need that RBAC cannot express. It leads into Lab 14.

Break it

Sign into Grafana as the local admin and change `Priya`'s org role to *Admin* by hand. Sign in as `Priya`: she is Admin. Sign out and back in: she is Editor again. What did Grafana do at login, and why is this behavior good for governance?

Reset: keep `Priya` as Dashboard Author; Lab 17 moves her. `./snapshot.sh lab13-end`.

LAB 14

Attribute-Based and Policy-Based Access Control

Prerequisites: Lab 13 · Artifact: attribute-based policies layered over the role model

Objective. Run Open Policy Agent as an external decision point and write policy as code. Roles set the baseline; attributes add the context roles can't express.

TASKS

1. Add OPA to `compose.yaml` and start it:

```
opa:
  image: openpolicyagent/opa:1.0.0
  command: ["run", "--server", "--addr=0.0.0.0:8181", "--watch", "/policies"]
  volumes: ["/policies:/policies"]
  ports: ["8181:8181"]
```

```
mkdir -p policies && docker compose up -d opa && curl -s localhost:8181/health
```

```
{}
```

2. Write the policy `policies/ledger.rego`. A role grants the baseline; each deny rule is an attribute condition that can take it away.

```

package halden.ledger

default allow := false

allow if {
    role_permits
    count(deny) == 0
}

role_permits if {
    input.action == "export_reports"
    "role-finance-analyst" in input.subject.roles
}

deny contains "outside business hours" if input.environment.hour < 8
deny contains "outside business hours" if input.environment.hour >= 18
deny contains "unmanaged device" if not input.device.managed
deny contains "restricted data needs MFA" if {
    input.resource.sensitivity == "restricted"
    input.subject.acr != "2"
}
deny contains "contractors cannot touch restricted data" if {
    input.resource.sensitivity == "restricted"
    input.subject.type == "contractor"
}

```

3. Ask for a decision. Save the request as q1.json and send it:

```

{"input": {"action": "export_reports",
  "subject": {"id": "pshah", "type": "employee", "roles": ["role-finance-analyst"],
  "acr": "1"},
  "resource": {"id": "q3-close", "sensitivity": "restricted"},
  "device": {"managed": true}, "environment": {"hour": 10}}}

```

```
curl -s localhost:8181/v1/data/halden/ledger -d @q1.json | jq .result
```

```
{ "allow": false, "deny": ["restricted data needs MFA"], "role_permits": true }
```

Change acr to "2" and send it again: allow becomes true. The role was never the problem. The context was.

4. Write policy tests in policies/ledger_test.rego. Cover one allow case and one case for each deny rule. Run them:

```
docker run --rm -v "$PWD/policies:/p" openpolicyagent/opa:1.0.0 test /p -v
```

```

data.halden.ledger_test.test_analyst_allowed: PASS
data.halden.ledger_test.test_after_hours_denied: PASS
...
PASS: 5/5

```

5. Add a relationship rule (ReBAC). Any user may read a report they own, whatever their role. Put ownership in `policies/data.json` as `{"owners": {"q3-close": "dcole3"}}` and add an allow rule that checks `data.owners[input.resource.id] == input.subject.id`. Test it as `dcole3`.
6. Write `abac-policies.md`. List each rule in plain English, which attribute it depends on, and which system owns that attribute. Use the data model from Lab 02.

DONE WHEN

- OPA returns a decision with reasons.
- All tests pass.
- Every attribute in the policy has an authoritative owner.

Break it

Remove the device object from `q1.json` entirely and send it again. Does the policy fail closed? Now change the `unmanaged-device` rule to `if input.device.managed == false` and repeat. Explain the difference. It is the most common ABAC mistake.

Reset: `restore not input.device.managed. ./snapshot.sh lab14-end` (policies live in the lab folder; commit them).

LAB 15

Entitlements, Permissions, and the Access Model

Prerequisites: Lab 13 · Artifact: entitlement catalog with birthright and requestable access

Objective. List every entitlement Grafana actually has, decide what everyone gets and what must be requested, and calculate one person's effective access across every system.

TASKS

1. Pull Grafana's access data from its API. Use the local admin account and the password you set on first login.

```
GA="admin:YOUR_GRAFANA_ADMIN_PASSWORD"
curl -s -u $GA localhost:3000/api/org/users | jq -r '.[ ] | "\(.login)\t\(.role)"'
curl -s -u $GA localhost:3000/api/folders | jq -r '.[ ].title'
```

```
admin    Admin
pshah    Editor
```

2. Create two folders, *Finance* and *Operations*, and restrict *Finance* so only Editors can edit it. Folder permissions are fine-grained entitlements that live only inside Grafana. The IdP knows nothing about them.
3. Write the catalog as notebook/lab15/catalog.yaml. Each entry needs an owner, a risk level, and a type: birthright or requestable.

```
- id: grafana.viewer
  system: Grafana
  grants: Org role Viewer – read all dashboards
  granted_by: LDAP group app-grafana-viewer
  type: birthright          # for sc-camel-department-number=Finance
  owner: dcole3
  risk: low
- id: grafana.editor
  grants: Org role Editor – create/edit dashboards, edit Finance folder
  granted_by: LDAP group app-grafana-editor
  type: requestable
  approver: owner
  max_duration: 180d
  risk: medium
- id: grafana.admin
  grants: Org role Admin – manage users, data sources, credentials
  granted_by: local grant only          # ungoverned today: finding
  type: privileged
  risk: high
```

Add entries for the intranet and for every Ledger entitlement from Lab 13.

4. Calculate effective access for Priya. This script gathers what each system says about her:

```
echo "== LDAP groups"; ldapsearch -x -LLL -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w
"$LDAP_ADMIN_PASSWORD" \
  -b ou=groups,dc=halden,dc=lab "(member=uid=pshah,ou=people,dc=halden,dc=lab)" cn | sed -n
's/^cn: //p'
echo "== Grafana role"; curl -s -u $GA "localhost:3000/api/org/users/lookup?query=pshah" | jq
-r '.[].role'
echo "== Ledger"; grep ^pshah notebook/lab13/ledger-access.csv
```

```
== LDAP groups
grp-finance
app-grafana-viewer
app-intranet-users
role-dashboard-author
app-grafana-editor
== Grafana role
Editor
== Ledger
pshah,Financial Analyst,view_reports;export_reports
```

Write out her effective access as a single list. Point out anything no single system would have shown you.

5. Define access bundles. Group entitlements so that a new Finance Analyst gets one bundle, not five separate requests. Also write down one entitlement that must never be part of a bundle, and why.

DONE WHEN

- Every Grafana, intranet, and Ledger entitlement is in the catalog with an owner and a risk level.
- Birthright and requestable access are clearly separated.
- Priya's effective access is written down, drawn from three sources.

Break it

Entitlement creep. Add Priya to grp-sales “temporarily for a project” and don't remove her. Run the effective-access script again. Which catalog field, had it been enforced, would have removed the access automatically? Leave the membership in place: Lab 17 cleans it up.

Reset: `./snapshot.sh lab15-end`. Keep the catalog; Labs 19 and 21 review against it.

LAB 16

Zero Trust and Modern Access Architecture

Prerequisites: Labs 10 and 14 · Artifact: conditional access policy set for the lab

Objective. Write conditional access policies that combine user, device, location, and risk into three outcomes: allow, step up, or block. Then remove the flat network that lets any container reach the directory.

TASKS

1. Write policies/access.rego:

```
package halden.access

default decision := "block"

decision := "allow" if {
    baseline_ok
    not needs_step_up
}

decision := "step_up" if {
    baseline_ok
    needs_step_up
}

baseline_ok if {
    trusted_device
    input.location.country in {"GB", "US", "IE"}
    input.risk < 70
}

trusted_device if {
    input.device.managed
    input.device.disk_encrypted
    input.device.os_patch_age_days <= 30
}

needs_step_up if input.risk >= 30
needs_step_up if {
    input.app.tier == "high"
    input.user.acr != "2"
}
```

2. Run six scenarios. Save scenarios.sh. Each scenario starts from Priya's posture file from Lab 10 and changes one thing.

```
base=$(jq -c '{device: ., user: {id: "pshah", acr: "1"}, location: {country: "GB"}, risk: 10, app: {tier: "low"}}' \
  notebook/lab10/posture-pshah.json)
try() { printf '%-28s ' "$1"; echo "$base" | jq -c "$2 | {input: .}" |
  curl -s localhost:8181/v1/data/halden/access/decision -d @- | jq -r .result; }
try "normal" '.'
try "high-tier app, acr 1" '.app.tier = "high"'
try "high-tier app, acr 2" '.app.tier = "high" | .user.acr = "2"'
try "unmanaged device" '.device.managed = false'
try "risk 80" '.risk = 80'
try "unexpected country" '.location.country = "XX"'
```

normal	allow
high-tier app, acr 1	step_up
high-tier app, acr 2	allow
unmanaged device	block
risk 80	block
unexpected country	block

3. Split the flat network. Add two networks to compose.yaml: idp for openldap and keycloak, and apps for keycloak, grafana, oauth2-proxy, whoami, and opa. Assign each service with a networks: list, then run `docker compose up -d`.

4. Prove the segmentation works:

```
docker run --rm --network halden_apps nicolaka/netshoot nc -zv -w 2 openldap 389
docker run --rm --network halden_idp nicolaka/netshoot nc -zv -w 2 openldap 389
```

```
nc: getaddrinfo for host "openldap" port 389: Name does not resolve ← apps can't see
the directory
Connection to openldap 389 port [tcp/ldap] succeeded!
```

5. Write conditional-access.md. Give each policy a plain-English statement, its signals, its outcome, and whether it is enforced today or report-only. Map each one to the zero trust components: policy engine (OPA), policy administrator (Keycloak session issuance), and enforcement points (oauth2-proxy, Grafana).
6. Write a three-step incremental roadmap: what Halden turns on next quarter, next half-year, and next year.

DONE WHEN

- All six scenarios give the expected decision.
- Apps can no longer reach LDAP directly, and SSO still works.
- Every policy lists its signals and its outcome.
- You have revised your Lab 01 concept map for the end of Part III.

Break it

Write a scenario where `risk` is missing from the input. What does the policy decide? Is that what you want? Then stop OPA (`docker compose stop opa`) and call the endpoint again. Nothing enforces these policies yet. Write down what a real PEP should do when the policy engine can't be reached, and for which apps.

Reset: `docker compose start opa. ./snapshot.sh lab16-end.`

Identity Governance

The lifecycle, automated provisioning, certification, compliance evidence, and governance that scales.

LAB 17

The Identity Lifecycle

Prerequisites: Lab 15 snapshot · Artifact: lifecycle process map with the access actions at each stage

Objective. Write a joiner-mover-leaver script and run it for one joiner, one mover (Priya, from Finance to Sales), and one leaver (Lee). Check at each stage that access changed exactly as intended, no more and no less.

TASKS

1. Create an OU for disabled accounts:

```
printf "dn: ou=disabled,dc=halden,dc=lab\nobjectClass: organizationalUnit\nou: disabled\n" | \
ldapadd -x -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w "$LDAP_ADMIN_PASSWORD"
```

2. Save `jm1.sh`. Birthright groups come from the department attribute, so they can always be explained.

```

#!/usr/bin/env bash
set -euo pipefail; source .env
BASE=dc=halden,dc=lab; B=(-x -H ldap://localhost -D cn=admin,$BASE -w "$LDAP_ADMIN_PASSWORD")
declare -A BIRTHRIGHT=([Finance]="grp-finance app-grafana-viewer" [Sales]="grp-sales"
                        [IT]="grp-it" [Operations]="grp-operations")
add_to() { if printf "dn: cn=$1,ou=groups,$BASE\nchangetype: modify\nadd: member\nmember:
$2\n" | ldapmodify "${B[@]}" >/dev/null 2>&1
        then echo " + $1"; else echo " = $1 (already a member)"; fi; }
del_from() { printf "dn: cn=$1,ou=groups,$BASE\nchangetype: modify\ndelete: member\nmember:
$2\n" | ldapmodify "${B[@]}" >/dev/null; echo " - $1"; }
groups_of(){ ldapsearch "${B[@]}" -LLL -b ou=groups,$BASE "(member=$1)" cn | sed -n 's/^cn:
//p'; }

case $1 in
joiner) # jml.sh joiner uid "First Last" dept title empno
        dn="uid=$2,ou=people,$BASE"; read -r fn ln <<< "$3"
        printf "dn: $dn\nobjectClass: inetOrgPerson\nuid: $2\ncn: $3\ngivenName: $fn\nsn: $ln\nmail:
$2@halden.lab\ndepartmentNumber: $4\ntitle: $5\nemployeeNumber: $6\nemployeeType: employee\n"
        | ldapadd "${B[@]}" >/dev/null
        echo "JOINER $2"; for g in app-intranet-users ${BIRTHRIGHT[$4]}; do add_to $g "$dn"; done ;;
mover) # jml.sh mover uid newdept
        dn="uid=$2,ou=people,$BASE"
        old=$(ldapsearch "${B[@]}" -LLL -b "$dn" -s base departmentNumber | sed -n
's/^departmentNumber: //p')
        echo "MOVER $2: $old -> $3"
        for g in ${BIRTHRIGHT[$old]}; do del_from $g "$dn"; done
        printf "dn: $dn\nchangetype: modify\nreplace: departmentNumber\ndepartmentNumber: $3\n" |
ldapmodify "${B[@]}" >/dev/null
        for g in ${BIRTHRIGHT[$3]}; do add_to $g "$dn"; done
        echo " ! still holds (needs review): $(groups_of "$dn" | grep -vE "app-intranet-
users|${BIRTHRIGHT[$3]}// ||}" | xargs) " ;;
leaver) # jml.sh leaver uid
        dn="uid=$2,ou=people,$BASE"; echo "LEAVER $2"
        for g in $(groups_of "$dn"); do del_from $g "$dn"; done
        ldappasswd "${B[@]}" "$dn" >/dev/null; echo " password scrambled"
        printf "dn: $dn\nchangetype: modify\nnewrdn: uid=$2\ndeleteolddn: 1\nnewsuperior:
ou=disabled,$BASE\n" | ldapmodify "${B[@]}" >/dev/null
        echo " moved to ou=disabled" ;;
esac

```

```

chmod +x jml.sh
./jml.sh joiner nkhan "Noor Khan" Finance "Accountant" 1201
./jml.sh mover pshah Sales
./jml.sh leaver ltran

```

```

JOINER nkhan
+ app-intranet-users
+ grp-finance
+ app-grafana-viewer
MOVER pshah: Finance -> Sales
- grp-finance
- app-grafana-viewer
= grp-sales (already a member)
! still holds (needs review): role-dashboard-author app-grafana-editor
LEAVER ltran
- grp-sales
- app-intranet-users
password scrambled
moved to ou=disabled

```

Notice two things. The script cannot decide on Priya's requested access by itself; it flags it for review. And her “temporary” grp-sales membership from Lab 15 (*already a member*) now looks legitimate, so nobody will question it.

3. Close Lee's active sessions. Moving his entry out of the directory stops new logins, but existing sessions carry on. In Keycloak, go to *Users* → *ltran* → *Sessions* and choose *Sign out*. Then run *User federation* → *ldap* → *Action* → *Sync changed users*.
4. Verify each person. Sign in as nkhan (set a password with `ldappasswd` first) and confirm Grafana shows Viewer. Sign in as pshah: her Grafana role is still Editor, from the flagged group. Sign in as ltran: *Invalid username or password*.
5. Resolve the mover review. Priya's new manager decides Sales doesn't need dashboard authoring. Remove her from `role-dashboard-author` and `app-grafana-editor` with `del_from`, or by hand. Grafana now refuses her login, because strict role mapping finds no rule for Sales. Write down whether that is correct, and what Sales should get instead.
6. Draw `lifecycle-map.png`. Use lanes for HR, the directory, the IdP, the apps, and the manager. For each stage (joiner, mover, leaver, plus the contractor's end date), list the trigger, the automatic actions, the manual decisions, and the verification step.

DONE WHEN

- Noor's access equals the birthright set, with nothing extra.
- Priya lost her Finance birthright access and her requested access was reviewed.
- Lee cannot sign in and has no active sessions.
- The process map covers the contractor, who has an end date and a sponsor.

Break it

Run `./jml.sh leaver` on a user who is the only member of a group, such as a new group `app-test` with only `mruiz`. The group removal fails. Find the LDAP error, explain which schema rule causes it, and fix the script so the leaver process doesn't stop halfway through.

Reset: if you tested with `mruiz`, restore `lab17-pre` or re-add him. Then `./snapshot.sh lab17-end`.

LAB 18

Provisioning and Identity Synchronization

Prerequisites: Lab 17; pip install ldap3 requests · Artifact: automated provisioning flow from the authoritative source to two applications

Objective. Make HR the trigger. A reconciliation script reads the HR file, works out what differs from the directory, and fixes it. Keycloak picks up the changes by synchronizing on a schedule, and Grafana is provisioned just in time and deprovisioned through its API.

TASKS

1. Update the HR export at notebook/lab18/hr.csv:

```
employee_id,uid,first,last,dept,title,status
1001,pshah,Priya,Shah,Sales,Sales Analyst,active
1002,mruiz,Marco,Ruiz,IT,Contractor - Migration,active
1003,dokafor,Dana,Okafor,IT,IT Operations Lead,active
1201,nkhan,Noor,Khan,Finance,Accountant,terminated
1202,sortiz,Sam,Ortiz,Operations,Warehouse Lead,active
```

2. Save sync.py. HR owns four attributes. The script never touches anything else, and it defaults to a dry run.

```

import csv, os, subprocess, sys, requests
from ldap3 import Server, Connection, MODIFY_REPLACE
BASE = "dc=halden,dc=lab"; PEOPLE = "ou=people," + BASE
DRY = "--apply" not in sys.argv
HR_OWNED = {"departmentNumber": "dept", "title": "title", "givenName": "first", "sn": "last"}
GRAFANA = ("admin", os.environ["GRAFANA_ADMIN_PASSWORD"])
c = Connection(Server("ldap://localhost"), "cn=admin," + BASE,
os.environ["LDAP_ADMIN_PASSWORD"], auto_bind=True)

def grafana_disable(uid):
    r = requests.get("http://localhost:3000/api/users/lookup", params={"loginOrEmail":
uid}, auth=GRAFANA)
    if r.ok: requests.post(f"http://localhost:3000/api/admin/users/{r.json()
['id']}/disable", auth=GRAFANA)
    return "disabled in Grafana" if r.ok else "no Grafana account"

for r in csv.DictReader(open(sys.argv[1])):
    uid = r["uid"]
    c.search(PEOPLE, f"(employeeNumber={r['employee_id']})", attributes=list(HR_OWNED))
    found = c.entries[0] if c.entries else None
    if r["status"] == "terminated":
        if found:
            print(f"LEAVER {uid}")
            if not DRY: subprocess.run(["./jml.sh", "leaver", uid], check=True); print("
" + grafana_disable(uid))
            continue
        if not found:
            print(f"JOINER {uid}")
            if not DRY: subprocess.run(["./jml.sh", "joiner", uid, f"{r['first']} {r['last']}",
r["dept"], r["title"], r["employee_id"]], check=True)
            continue
    have = found.entry_attributes_as_dict
    for attr, col in HR_OWNED.items():
        cur = (have.get(attr) or [""])[0]
        if cur != r[col]:
            print(f"UPDATE {uid}: {attr} '{cur}' -> '{r[col]}'")
            if not DRY: c.modify(found.entry_dn, {attr: [(MODIFY_REPLACE, [r[col]])]})
    print("DRY RUN, nothing changed" if DRY else "APPLIED")

```

Add GRAFANA_ADMIN_PASSWORD to .env first.

- Before Noor's account is removed, have her sign into Grafana once so that JIT provisioning creates her account there. Check it exists: `curl -s -u admin:$GRAFANA_ADMIN_PASSWORD`

`"localhost:3000/api/users/lookup?loginOrEmail=nkhan" | jq .login → "nkhan".`

- Do a dry run, read the plan, then apply:

```

set -a; source .env; set +a
python3 sync.py notebook/lab18/hr.csv

```

```
UPDATE pshah: title 'Financial Analyst' -> 'Sales Analyst'
UPDATE dokafor: title 'Systems Administrator' -> 'IT Operations Lead'
LEAVER nkhan
JOINER sortiz
DRY RUN, nothing changed
```

```
python3 sync.py notebook/lab18/hr.csv --apply
python3 sync.py notebook/lab18/hr.csv          # run again: should find nothing to do
```

```
DRY RUN, nothing changed
```

The second run finding nothing proves the script is idempotent: running it twice has the same effect as running it once.

5. Let Keycloak synchronize on a schedule. Go to *User federation* → *ldap* → *Synchronization settings*, turn on *Periodic changed users sync* every 60 seconds, and turn on *Periodic full sync* every 3600 seconds. Wait a minute, then check that Priya's user page in Keycloak shows the title *Sales Analyst*.
6. Schedule the HR sync too, with cron (`* /15 * * * *`) or a systemd timer. Log the output to `logs/sync-YYYYMMDD.log`. Lab 20 uses these logs as evidence.
7. Draw provisioning-flow.png: HR → sync.py → OpenLDAP → Keycloak (periodic sync) → Grafana (JIT at login, API disable on leave) and intranet (checked at every login). For each attribute, label its source of authority.

DONE WHEN

- The dry run matches, the applied run works, and a second run finds nothing to do.
- Noor is disabled in both the directory and Grafana.
- Keycloak shows the HR title change within 60 seconds.
- The flow diagram names the source of authority for each attribute.

Break it

The silent failure. Create a directory user with no HR record: `./jml.sh joiner ghost "Ghost User" Sales "Sales Rep" 9999`. Then run the sync. It reports nothing. Add a reconciliation pass that lists every person in `ou=people` whose `employeeNumber` is missing from HR. What does it find among the 35 generated users, and what should Halden do about them?

Reset: `run ./jml.sh leaver ghost. ./snapshot.sh lab18-end.`

LAB 19

Identity Governance and Administration

Prerequisites: Labs 15 and 18 · Artifact: access certification campaign with meaningful reviewer decisions

Objective. Generate a certification campaign that gives reviewers the context they need. Run the reviews, apply the revocations automatically, and add an access-request check that blocks toxic combinations before they are granted.

TASKS

1. Get a Keycloak admin token. You will reuse this helper in later labs; save it as `kc_token.sh`.

```
export KC=$(curl -s http://keycloak:8080/realms/master/protocol/openid-connect/token \
  -d grant_type=password -d client_id=admin-cli -d username=admin -d
  password="$KC_ADMIN_PASSWORD" | jq -r .access_token)
```

2. Build the campaign. For each member of each access group, the script records the entitlement, the risk from the catalog, the reviewer (the entitlement owner), and the date of the user's last login. Save as `campaign.py`:

```
import os, csv, sys, requests
from ldap3 import Server, Connection
KC = {"Authorization": "Bearer " + os.environ["KC"]}; API =
"http://keycloak:8080/admin/realms/halden"
RISK = {"app-grafana-editor": "medium", "grp-lab-admins": "high", "app-grafana-viewer": "low"}
OWNER = {"app-grafana-editor": "dcole3", "app-grafana-viewer": "dcole3", "grp-lab-
admins": "ciso"}
def last_login(uid):
    ev = requests.get(f"{API}/events", params={"type": "LOGIN", "max": 1, "user":
user_id(uid)}, headers=KC).json()
    return __import__("datetime").date.fromtimestamp(ev[0]["time"] / 1000).isoformat() if ev
    else "never"
def user_id(uid):
    u = requests.get(f"{API}/users", params={"username": uid, "exact":
"true"}, headers=KC).json()
    return u[0]["id"] if u else ""
c = Connection(Server("ldap://localhost"), "cn=admin,dc=halden,dc=lab",
os.environ["LDAP_ADMIN_PASSWORD"], auto_bind=True)
w = csv.writer(sys.stdout); w.writerow(["entitlement", "risk", "user", "title", "last_login",
"reviewer", "decision", "reason"])
for g in RISK:
    c.search(f"cn={g},ou=groups,dc=halden,dc=lab", "(objectClass=*)", attributes=["member"])
    for dn in c.entries[0].member.values:
        c.search(dn, "(objectClass=*)", attributes=["uid", "title"]); e = c.entries[0]
        w.writerow([g, RISK[g], e.uid, e.title, last_login(str(e.uid)), OWNER[g], "", ""])
```

```
source kc_token.sh; python3 campaign.py > notebook/lab19/campaign-2026Q3.csv; column -s, -t <
notebook/lab19/campaign-2026Q3.csv | head -6
```

entitlement	risk	user	title	last_login	reviewer
decision reason					
app-grafana-editor	medium	...			
grp-lab-admins	high	dokafor	IT Operations Lead	2026-09-24	ciso
app-grafana-viewer	low	ang0	Accountant	never	dcole3
...					

3. Be the reviewer. For every row, fill in decision as keep or revoke, and give a reason. Do not approve anything whose last login is *never* without writing down why it is still needed.
4. Apply the revocations. Write `apply_revocations.py`: for each row where the decision is revoke, remove the member from the group with `ldap3 (MODIFY_DELETE)` and print a receipt line with a timestamp. Save the receipts as `revocations-2026Q3.log`.
5. Add a check before granting a request. Save `request_check.py`, which refuses any request that would create a toxic pair from your Lab 13 matrix:

```
TOXIC = [{"ledger:create_vendor", "ledger:approve_payment"},
{"ledger:create_invoice", "ledger:approve_payment"}]
def check(current, requested):
    after = set(current) | {requested}
    hits = [p for p in TOXIC if p <= after]
    return ("DENY: SoD conflict " + " + ".join(sorted(hits[0]))) if hits else "OK: route to
owner for approval"
print(check(["ledger:view_reports", "ledger:create_vendor"], "ledger:approve_payment"))
print(check(["ledger:view_reports"], "ledger:export_reports"))
```

```
DENY: SoD conflict ledger:approve_payment + ledger:create_vendor
OK: route to owner for approval
```

6. Write `campaign-design.md`. Cover scope, reviewer assignment, the context each reviewer sees, deadlines and escalation, what happens when a reviewer doesn't respond (the default must be revoke, not keep), and the metrics that would reveal rubber-stamping.

DONE WHEN

- The campaign CSV has risk, reviewer, and last-login context for every row.
- Every row has a decision and a reason.
- Revocations are applied and receipts are logged.
- `request_check.py` denies the toxic pair.

Break it

Self-granting. Change the reviewer of `app-grafana-editor` to `pshah` and regenerate the campaign. Priya is now reviewing her own access. Add a rule to `campaign.py` that reassigns any self-review to the user's manager, using the `manager` attribute from Lab 04.

Reset: `./snapshot.sh lab19-end`. Keep the campaign CSV and revocation log; Lab 20 uses them as evidence.

LAB 20

Compliance, Audit, and Regulatory Drivers

Prerequisites: Lab 19 · Artifact: control-to-requirement mapping for one framework

Objective. Map the lab's controls to ISO/IEC 27001:2022 Annex A and produce an evidence pack with one command, so that answering the auditor's four questions is a report, not a scramble.

TASKS

1. Make sure Keycloak keeps its evidence. Under *Realm settings* → *Events*, turn on *Admin events* and *Include representation*, and set user event expiration to 400 days. Note that the lab's H2 database is not durable storage for audit records; write down where production logs should go.
2. Write `control-map.md` with the columns *requirement · Halden control · lab mechanism · evidence artifact · owner · frequency*. At minimum, cover these Annex A controls: 5.15 Access control, 5.16 Identity management, 5.17 Authentication information, 5.18 Access rights, 8.2 Privileged access rights, 8.5 Secure authentication, 8.15 Logging. Example row:

```
5.18 Access rights | Quarterly certification of access groups | campaign.py +  
apply_revocations.py |  
campaign-YYYYQn.csv, revocations-YYYYQn.log | IAM lead | quarterly
```

3. Build the evidence pack. Save `evidence.sh`:

```
#!/usr/bin/env bash  
set -euo pipefail; source .env; source kc_token.sh  
OUT=evidence/$(date +%F); mkdir -p $OUT; B=(-x -H ldap://localhost -D  
cn=admin,dc=halden,dc=lab -w "$LDAP_ADMIN_PASSWORD")  
# Q1 who has access  
ldapsearch "${B[@]}" -LLL -b ou=groups,dc=halden,dc=lab "(objectClass=groupOfNames)" cn member  
modifyTimestamp > $OUT/q1-access.ldif  
# Q2 who approved it / Q3 when reviewed  
cp notebook/lab19/campaign-*.csv $OUT/ 2>/dev/null || true  
# Q4 when removed  
cp notebook/lab19/revocations-*.log logs/sync-*.log $OUT/ 2>/dev/null || true  
ldapsearch "${B[@]}" -LLL -b ou=disabled,dc=halden,dc=lab uid modifyTimestamp >  
$OUT/q4-disabled.ldif  
# authentication and admin activity  
curl -s -H "Authorization: Bearer $KC" "http://keycloak:8080/admin/realms/halden/events?  
max=1000" > $OUT/auth-events.json  
curl -s -H "Authorization: Bearer $KC" "http://keycloak:8080/admin/realms/halden/admin-events?  
max=1000" > $OUT/admin-events.json  
curl -s -H "Authorization: Bearer $KC" "http://keycloak:8080/admin/realms/halden" | jq  
'{passwordPolicy, bruteForceProtected, otpPolicyType}' > $OUT/auth-config.json  
sha256sum $OUT/* > $OUT/MANIFEST.sha256; ls $OUT
```

```
MANIFEST.sha256  admin-events.json  auth-config.json  auth-events.json  campaign-2026Q3.csv  
q1-access.ldif  q4-disabled.ldif  revocations-2026Q3.log  sync-20260924.log
```

4. Answer the auditor's four questions for Lee Tran, using only the evidence pack: who had access, who approved it, when it was reviewed, and when it was removed. Where the evidence runs out, write *gap* and name the control that would fill it.
5. Demonstrate separation of duties. Show from the evidence that no one holds both `app-grafana-editor` and `grp-lab-admins`, or whichever toxic pair you defined. Write the `jq` or `grep` command an auditor could rerun.

DONE WHEN

- ☐ All seven Annex A controls are mapped to mechanisms and artifacts.
- ☐ `evidence.sh` runs in one step and writes a hash manifest.
- ☐ The Lee Tran audit trail is answered, with gaps named.
- ☐ Each control is labeled continuous or point-in-time.

Break it

Edit one line in `q1-access.ldif` after the pack is generated, then run `sha256sum -c MANIFEST.sha256`. What does the output show? What would stop an administrator from regenerating the manifest after tampering, and where should the manifest be stored instead?

Reset: regenerate the pack. Add `evidence/` to `.gitignore` if it contains real-looking data.
`./snapshot.sh lab20-end`.

LAB 21

Access Reviews, Analytics, and Intelligent Governance

Prerequisites: Lab 19 · Artifact: risk-based review model that prioritizes high-risk access

Objective. Replace “review everything” with a model that scores each access by risk. Flag access that doesn't match the person's peers or hasn't been used, and send only the top items to a human.

TASKS

1. Use this fixed sample so your results match the expected output. Save as notebook/lab21/access.csv. The columns are user, department, entitlement, entitlement risk (1–3), and days since the access was last used.

```
user,dept,entitlement,risk,days_unused
ang0,Finance,grafana.viewer,1,3
bpark1,Finance,grafana.viewer,1,5
cdiaz2,Finance,grafana.viewer,1,2
cdiaz2,Finance,ledger.approve_payment,3,4
emoss4,Finance,grafana.viewer,1,1
pshah,Sales,grafana.viewer,1,40
pshah,Sales,crm.user,1,1
tsato5,Sales,crm.user,1,2
vrao6,Sales,crm.user,1,3
vrao6,Sales,ledger.approve_payment,3,190
dokafor,IT,grp-lab-admins,3,1
mruiiz,IT,grp-lab-admins,3,95
```

2. Save score.py. A score rises with entitlement risk, with rarity among the person's department peers, and with how long the access has gone unused.

```
import csv, collections
rows = list(csv.DictReader(open("access.csv")))
people = collections.defaultdict(set); holders = collections.Counter()
for r in rows: people[r["dept"]].add(r["user"]); holders[(r["dept"], r["entitlement"])] += 1
out = []
for r in rows:
    peer_share = holders[(r["dept"], r["entitlement"])] / len(people[r["dept"]])
    outlier = peer_share < 0.5
    stale = int(r["days_unused"]) > 90
    score = int(r["risk"]) * 10 + (15 if outlier else 0) + (20 if stale else 0)
    out.append((score, r["user"], r["entitlement"], f"peers {peer_share:.0%}", "STALE" if
    stale else ""))
for s in sorted(out, reverse=True)[:5]: print(*s, sep=" ")
print("auto-certify (score < 20):", sum(1 for s in out if s[0] < 20), "of", len(out))
```

```
cd notebook/lab21 && python3 score.py
```

```
65 vrao6 ledger.approve_payment peers 33% STALE
50 mruiz grp-lab-admins peers 100% STALE
45 cdiaz2 ledger.approve_payment peers 25%
30 dokafor grp-lab-admins peers 100%
25 pshah grafana.viewer peers 33%
auto-certify (score < 20): 7 of 12
```

3. Read the results as a reviewer. vrao6 in Sales holds a Finance payment right they haven't used in six months. Marco, a contractor, holds admin rights he hasn't used in 95 days. Priya's viewer access is left over from before her move. Write a micro-certification for each: one question and one yes/no decision, with the evidence attached.
4. Notice that mruiz scores 100% peer share because both IT members hold admin rights. Write two sentences on why peer-group analysis fails when the whole group is over-privileged. Add a rule that fixes it, for example that risk-3 entitlements always go to human review.
5. Policy mining. Compare the observed access in `access.csv` with the birthright rules in `jm1.sh`. List every entitlement that the written policy does not explain.
6. Write `risk-review-model.md`. Include the scoring formula, the thresholds (auto-certify, micro-certify, full review), the unused-access revocation rule with a notice period, and the limits of letting a model decide instead of a person.

DONE WHEN

- ☐ Your output matches, and fewer than half the rows need a human.
- ☐ Three micro-certifications are written, each with its evidence.
- ☐ The model has a safeguard for groups that are over-privileged as a whole.
- ☐ You have revised your Lab 01 concept map for the end of Part IV.

Break it

Gaming the metric. Change every `days_unused` value to 1, as if a script logged in to every app each night. Rerun. What does the model now miss? Which independent signal would expose that kind of fake activity?

Reset: restore the original CSV. No lab state changed.

Privileged and Non-Human Identity

Vaulting, just-in-time elevation, privilege paths, and the service accounts that outnumber people.

LAB 22

Privileged Access Management

Prerequisites: Lab 21 snapshot · Artifact: privileged access model with vaulting, session control, and JIT elevation

Objective. Give Dana a separate admin account whose password only the vault knows and rotates. Remove all standing admin membership, add time-limited elevation tied to a ticket, and record privileged sessions.

TASKS

1. Add a vault to `compose.yaml`. Dev mode keeps everything in memory, which is fine for the lab and never acceptable in production. OpenBao (`openbao/openbao`, command `bao`) works the same way if you prefer a fully open-source licence.

```
vault:
  image: hashicorp/vault:1.17
  command: server -dev
  environment: {VAULT_DEV_ROOT_TOKEN_ID: root, VAULT_DEV_LISTEN_ADDRESS: "0.0.0.0:8200"}
  cap_add: [IPC_LOCK]
  ports: ["8200:8200"]
  networks: [idp, apps]
```

```
docker compose up -d vault
v() { docker compose exec -e VAULT_ADDR=http://127.0.0.1:8200 -e VAULT_TOKEN=root vault vault "$@"; }
v status | grep Sealed
```

Sealed	false
--------	-------

2. Create the admin account and a placeholder member. A `groupOfNames` must always have at least one member (see the Lab 17 answer), so the placeholder keeps the admin group valid once it has no real members.

```

ldapadd -x -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w "$LDAP_ADMIN_PASSWORD" <<EOF
dn: ou=admins,dc=halden,dc=lab
objectClass: organizationalUnit
ou: admins

dn: uid=adm-dokafor,ou=admins,dc=halden,dc=lab
objectClass: inetOrgPerson
uid: adm-dokafor
cn: Dana Okafor (admin)
sn: Okafor
employeeType: privileged
seeAlso: uid=dokafor,ou=people,dc=halden,dc=lab

dn: uid=placeholder,ou=service,dc=halden,dc=lab
objectClass: inetOrgPerson
uid: placeholder
cn: Group placeholder
sn: none
EOF

```

3. Vault the admin password. The LDAP secrets engine sets a random password now and rotates it every 24 hours.

```

v secrets enable ldap
v write ldap/config binddn=cn=admin,dc=halden,dc=lab bindpass="$LDAP_ADMIN_PASSWORD" \
  url=ldap://openldap:389 schema=openldap
v write ldap/static-role/adm-dokafor username=adm-dokafor \
  dn=uid=adm-dokafor,ou=admins,dc=halden,dc=lab rotation_period=24h
v read ldap/static-cred/adm-dokafor

```

Key	Value
dn	uid=adm-dokafor,ou=admins,dc=halden,dc=lab
last_vault_rotation	2026-09-24T10:12:03Z
password	Hk2...q9 ← no human chose it or knows it until checkout
rotation_period	24h
ttl	23h59m52s
username	adm-dokafor

Force a rotation with `v write -f ldap/rotate-role/adm-dokafor`, then confirm the old password no longer binds and the new one does.

4. Remove standing privilege. Add the placeholder to `grp-lab-admins`, then remove `dokafor`. The group now has no human members at all.
5. Save `elevate.sh` for just-in-time elevation:

```
#!/usr/bin/env bash
set -euo pipefail; source .env; mkdir -p logs
[ $# -eq 3 ] || { echo "usage: elevate.sh ACCOUNT MINUTES TICKET"; exit 1; }
BASE=dc=halden,dc=lab; B=(-x -H ldap://localhost -D cn=admin,$BASE -w "$LDAP_ADMIN_PASSWORD")
DN="uid=$1,ou=admins,$BASE"; G="cn=grp-lab-admins,ou=groups,$BASE"
mod() { printf "dn: $G\nchangetype: modify\n$1: member\nmember: $DN\n" | ldapmodify "${B[@]}"
>/dev/null; }
mod add; echo "$(date -Is) GRANT $1 grp-lab-admins ${2}m ticket=$3" | tee -a logs/jit.log
( sleep $(( $2 * 60 )); mod delete; echo "$(date -Is) REVOKE $1 grp-lab-admins expired" >>
logs/jit.log ) &
disown
```

```
chmod +x elevate.sh; ./elevate.sh adm-dokafor 2 CHG-1042
sleep 130; cat logs/jit.log
```

```
2026-09-24T10:20:00+00:00 GRANT adm-dokafor grp-lab-admins 2m ticket=CHG-1042
2026-09-24T10:22:00+00:00 REVOKE adm-dokafor grp-lab-admins expired
```

6. Record a privileged session. On Linux, wrap the admin shell with `script`. On macOS, `script -r` records and `script -p` plays back.

```
mkdir -p sessions; script -q -t 2>sessions/CHG-1042.timing sessions/CHG-1042.log
# ... run your admin ldapmodify commands, then type: exit
scriptreplay sessions/CHG-1042.timing sessions/CHG-1042.log
```

7. Write `pam-model.md`. Cover the inventory of privileged accounts (LDAP `cn=admin`, Keycloak master admin, Vault root, `adm-dokafor`, the Grafana admin), plus for each account: its vaulting, rotation, elevation path, maximum duration, session recording, and approver. Add a break-glass procedure: which account, where its credential is kept, who can open it, and what alerts fire when it is used.

DONE WHEN

- No human is a standing member of `grp-lab-admins`.
- The admin password is vaulted, and after rotation the old one fails.
- JIT elevation grants and expires, and both events are logged with a ticket number.
- A recorded session replays.

Break it

Run `docker compose restart vault`, then `v read ldap/static-cred/adm-dokafor`. The dev vault has lost everything, and `adm-dokafor`'s password is now unknown to anyone. How do you recover? What does this teach you about the vault's availability being part of the break-glass design?

Reset: the dev vault forgets everything on restart. Re-run tasks 3 and 5 after any restart, or restore `lab21-end` to start over. `./snapshot.sh lab22-end`.

LAB 23

Privileged Access Attacks and Defenses

Prerequisites: Lab 22; pip install networkx · Artifact: tiered administration design with detection points

Objective. Think in graphs. Find every path from an ordinary account to control of the IdP, look at how exposed the lab's own secrets are, and write a detection that fires when someone escalates. Everything in this lab is defensive and stays inside your lab.

TASKS

1. Model who can control what. This graph reflects the lab as it stood at the end of Lab 13, with a few realistic weaknesses added. Save as notebook/lab23/edges.csv:

```
src,rel,dst
dokafor,Owns,adm-dokafor
mruiz,MemberOf,grp-it
grp-it,CanResetPassword,adm-dokafor
adm-dokafor,MemberOf,grp-lab-admins
grp-lab-admins,AdminOf,keycloak
pshah,MemberOf,app-grafana-editor
app-grafana-editor,CanEditDatasource,grafana
grafana,StoresSecretOf,svc-reporting
svc-reporting,BindsAs,cn=admin
cn=admin,AdminOf,openldap
openldap,FederatedInto,keycloak
```

2. Find the paths to the crown jewel with paths.py:

```
import csv, networkx as nx
G = nx.DiGraph()
for r in csv.DictReader(open("edges.csv")): G.add_edge(r["src"], r["dst"], rel=r["rel"])
for start in ["pshah", "mruiz", "dokafor", "svc-reporting"]:
    try:
        p = nx.shortest_path(G, start, "keycloak")
        print(f"{start:14} {len(p)-1} hops " + " -> ".join(p))
    except nx.NetworkXNoPath:
        print(f"{start:14} no path")
```

```
pshah          6 hops  pshah -> app-grafana-editor -> grafana -> svc-reporting -> cn=admin ->
openldap -> keycloak
mruiz          4 hops  mruiz -> grp-it -> adm-dokafor -> grp-lab-admins -> keycloak
dokafor        3 hops  dokafor -> adm-dokafor -> grp-lab-admins -> keycloak
svc-reporting  3 hops  svc-reporting -> cn=admin -> openldap -> keycloak
```

A list of group memberships would never show you that a dashboard editor is six steps from owning the IdP. For each path, pick the one edge whose removal breaks it at the lowest cost.

3. Look at how secrets are exposed on the admin plane. Anyone who controls the Docker host can read every secret in the lab:

```
docker compose exec grafana env | grep CLIENT_SECRET
docker inspect halden-oauth2-proxy-1 --format '{{json .Args}}' | tr ',' '\n' | grep secret
```

```
GF_AUTH_GENERIC_OAUTH_CLIENT_SECRET=8fJ2...
"--client-secret=Qa91..."
"--cookie-secret=..."
```

This is why the Docker host belongs in tier 0 alongside the IdP. Record it in your tier design.

4. Write a detection. OpenLDAP's default log level (256, called “stats”) records every modification. Elevate Dana again, then search the log:

```
./elevate.sh adm-dokafor 1 CHG-1043
docker compose logs openldap --since 5m | grep -A1 'MOD dn="cn=grp-lab-admins'
```

```
... conn=1087 op=1 MOD dn="cn=grp-lab-admins,ou=groups,dc=halden,dc=lab"
... conn=1087 op=1 MOD attr=member
```

Save `detect.sh`. It should compare each admin-group change with `logs/jit.log` and print ALERT for any change that has no matching ticketed grant. Test it by adding a member by hand with `ldapmodify`.

5. Write `tiering.md`. Tier 0 is the identity plane: OpenLDAP, Keycloak, Vault, and the Docker host. Tier 1 is the apps: Grafana, the intranet, OPA. Tier 2 is user workstations. State the rule that no credential from a higher tier is ever used on a lower-tier system. Describe a privileged access workstation for tier-0 work, and mark where each detection sits.

DONE WHEN

- All four paths are found, and a cheap edge to cut is chosen for each.
- `detect.sh` stays quiet for the ticketed grant and alerts on the manual one.
- The tier model places the Docker host in tier 0 and explains why.

Break it

Remove the edge `grp-it`, `CanResetPassword`, `adm-dokafor` and rerun `paths.py`. Marco's path disappears. Now add `mruiz`, `MemberOf`, `app-grafana-editor`. How many hops does he need now, and what does this tell you about fixing one path at a time?

Reset: remove any manual admin-group members you added. `./snapshot.sh lab23-end`.

LAB 24

Service Accounts and Non-Human Identity

Prerequisites: Lab 23 · Artifact: non-human identity governance policy with lifecycle rules by type

Objective. Inventory every non-human identity in the lab. Replace Keycloak's all-powerful directory bind with a read-only service account, and give the reporting service a short-lived token instead of a password that never changes.

TASKS

1. Take inventory. List Keycloak's confidential clients and the directory's service accounts:

```
source kc_token.sh
curl -s -H "Authorization: Bearer $KC" http://keycloak:8080/admin/realms/halden/clients \
  | jq -r '.[ ] | select(.publicClient == false) | .clientId'
ldapsearch -x -LLL -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w
"$LDAP_ADMIN_PASSWORD" \
  -b ou=service,dc=halden,dc=lab uid description
```

Your client list includes grafana, intranet, and cli-test, plus built-in clients such as broker and realm-management. Record every one in `nhi-inventory.md`, along with the non-human secrets in `.env` (the cookie secret, the Vault root token). Give each an owner, a credential type, where the credential is stored, when it was last rotated, and its privileges.

2. Create a read-only bind account for Keycloak:

```
printf "dn: uid=svc-keycloak,ou=service,dc=halden,dc=lab\nobjectClass: inetOrgPerson\nuid:
svc-keycloak\ncn: Keycloak LDAP bind\nnsn: Service\ndescription: Owner: dokafor; expires: 2027-
09-24\n" \
  | ldapadd -x -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w "$LDAP_ADMIN_PASSWORD"
SVC_KC_PW=$(openssl rand -base64 24); echo "SVC_KC_PW=$SVC_KC_PW" >> .env
ldappasswd -x -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w "$LDAP_ADMIN_PASSWORD" -s
"$SVC_KC_PW" uid=svc-keycloak,ou=service,dc=halden,dc=lab
```

3. Grant it read access and nothing more. Directory ACLs live in `cn=config`, which you edit from inside the container:

```
docker compose exec -T openldap ldapmodify -Y EXTERNAL -H ldapi:/// <<EOF
dn: sc-camel-olc-database={1}mdb,cn=config
changetype: modify
add: olcAccess
olcAccess: {0}to dn.subtree="dc=halden,dc=lab" attrs=userPassword by self write by anonymous
auth by * none
olcAccess: {1}to dn.subtree="dc=halden,dc=lab" by dn.exact="uid=svc-
keycloak,ou=service,dc=halden,dc=lab" read by * break
EOF
```

```
K=(-x -H ldap://localhost -D uid=svc-keycloak,ou=service,dc=halden,dc=lab -w "$SVC_KC_PW")
ldapsearch "${K[@]}" -LLL -b ou=people,dc=halden,dc=lab "(uid=pshah)" cn userPassword
printf "dn: uid=pshah,ou=people,dc=halden,dc=lab\nchangetype: modify\nreplace: title\ntitle:
CEO\n" | ldapmodify "${K[@]}"
```

```
dn: uid=pshah,ou=people,dc=halden,dc=lab
cn: Priya Shah                                ← readable; userPassword is not returned
ldap_modify: Insufficient access (50)          ← cannot write
```

4. Switch Keycloak to the new account. In *User federation* → *ldap*, set Bind DN to uid=svc-keycloak,ou=service,dc=halden,dc=lab and enter its password. Click *Test connection* and *Test authentication*, both should succeed. Then sign into the intranet as pshah. Mark the Lab 05 finding as closed.
5. Give the reporting service a workload identity. Create the client svc-reporting with client authentication on, *Service accounts roles* ticked, and every other flow off. Under *Advanced* → *Access Token Lifespan*, set 5 minutes.

```
curl -s http://keycloak:8080/realms/halden/protocol/openid-connect/token -d
grant_type=client_credentials \
-d client_id=svc-reporting -d client_secret="$SVC_REPORTING_SECRET" \
| jq -r .access_token | step crypto jwt inspect --insecure | jq '.payload | {sub, azp, exp,
iat, preferred_username}'
```

```
{ "sub": "c41e...", "azp": "svc-reporting", "exp": 1790251500, "iat": 1790251200,
  "preferred_username": "service-account-svc-reporting" }
```

The token expires in 300 seconds. Even the client secret that obtains it can be rotated by Vault or Keycloak on a schedule.

6. Rotate the old directory password for svc-reporting hourly, using the Vault static role from Lab 22 (rotation_period=1h).
7. Write nhi-policy.md. For each type (service account, OAuth client, API key, workload token, bot), give a mandatory owner, a maximum credential lifetime, a rotation method, a discovery source, an expiry date that is set by default, and the retirement trigger: owner leaves, unused for 90 days, or expiry reached.

DONE WHEN

- Keycloak binds as svc-keycloak, which can read but not write.
- svc-reporting gets a 5-minute token through client credentials.
- Every non-human identity in the inventory has an owner and an expiry date.
- You have revised your Lab 01 concept map for the end of Part V.

Break it

Orphan the owner. Run `./jml.sh leaver dokafor` in a snapshot. `svc-keycloak` keeps working, but its owner no longer exists. Write the query that finds every non-human identity whose description owner is not an active person, and add it to the NHI policy as a weekly control.

Reset: `restore pre-break` to bring Dana back. `./snapshot.sh lab24-end`.

Federation and Beyond the Enterprise

Trust across boundaries, identity for customers, and credentials the holder carries.

LAB 25

Federation and Cross-Domain Identity

Prerequisites: Lab 24 snapshot; xmllint · Artifact: working federation between the lab IdP and an application using a standard protocol

Objective. Let a user from a partner organization reach the Halden intranet without a Halden account, using SAML. You will read the signed assertion, map the partner's claims to local access, and break the trust.

TASKS

1. Set up the partner IdP. Create a realm named `partner` and a user `alex` with email `alex@partner.example`, first and last name, and a password.
2. Make Halden trust the partner. In the `halden` realm, go to *Identity providers* → *SAML v2.0*. Set alias `partner-saml` and display name *Partner Co*, then paste the SAML entity descriptor `http://keycloak:8080/realms/partner/protocol/saml/descriptor`. Set NameID policy format to *Email*. Turn on *Want AuthnRequests signed*, *Validate signatures*, and *Use metadata descriptor URL*. Save.
3. Make the partner trust Halden. Export Halden's service-provider metadata and import it as a client in the partner realm:

```
curl -s http://keycloak:8080/realms/halden/broker/partner-saml/endpoint/descriptor -o notebook/lab25/halden-sp.xml
grep -o 'entityID="[^\"]*"' notebook/lab25/halden-sp.xml
```

```
entityID="http://keycloak:8080/realms/halden"
```

In the `partner` realm, choose *Clients* → *Import client*, upload `halden-sp.xml`, and save. On the new client, set Name ID format to *email* and turn on *Force name ID format*.

4. Try it. In a private window, open `http://localhost:4180`. The Halden login page now has a *Partner Co* button. Sign in as `alex` and complete the first-login profile review. You land on **403 Forbidden**: authentication was federated, but authorization is still local.
5. Map claims to access. Create a Keycloak group `partner-guests` in `halden`. On the `partner-saml` provider, choose *Mappers* → *Add mapper*, type *Hardcoded group*, group `partner-guests`. Add `--allowed-group=partner-guests` to `oauth2-proxy` (the flag can be repeated) and recreate the container. Sign in as `alex` again: the intranet loads.

6. Read the assertion. In DevTools, find the POST to `/realms/halden/broker/partner-saml/endpoint` and copy the SAMLResponse form value:

```
pbpaste | python3 -c "import sys,urllib.parse;print(urllib.parse.unquote(sys.stdin.read()))" |
base64 -d | xmllint --format - \
| grep -E "Issuer|NameID|Audience|NotOnOrAfter|SignatureValue" | head # Linux: xclip -o
instead of pbpaste
```

```
<saml:Issuer>http://keycloak:8080/realms/partner</saml:Issuer>
<dsig:SignatureValue>k2F...</dsig:SignatureValue>
<saml:NameID Format="urn:oasis:names:tc:SAML:1.1:nameid-
format:emailAddress">alex@partner.example</saml:NameID>
<saml:Conditions NotBefore="2026-09-24T11:00:00Z" NotOnOrAfter="2026-09-24T11:01:00Z">
<saml:Audience>http://keycloak:8080/realms/halden</saml:Audience>
```

Match each line to what it protects against: the issuer and signature against forgery, the audience against replay at a different SP, and the time window against replay later.

7. Write `federation.md`. Cover what each side vouches for, the metadata exchange, the claim mapping, and a comparison with OIDC: the OIDC login to Grafana in Lab 06, and the OAuth-only client credentials token in Lab 24, which grants access but identifies no user.

DONE WHEN

- alex reaches the intranet through SAML, and has no LDAP account.
- You have decoded the assertion and annotated the issuer, NameID, audience, conditions, and signature.
- The document distinguishes SAML, OAuth, and OIDC by purpose.

Break it

Rotate the partner's signing key. In the partner realm, go to *Realm settings* → *Keys* → *Add provider* → *rsa-generated* with priority 200, then disable the old *rsa-generated* key. Sign in as alex. Then turn off *Use metadata descriptor URL* on the Halden side and repeat. Record what happens in each case, and the Keycloak event you see.

Reset: turn *Use metadata descriptor URL* back on, and choose *Reload keys* on the provider if needed.
./snapshot.sh lab25-end.

LAB 26

Customer and Consumer Identity

Prerequisites: Lab 25 · Artifact: consumer registration and authentication design balancing friction and security

Objective. Build a customer realm for Halden's web shop. It covers self-registration with email verification, consent, terms acceptance, and progressive profiling that asks for a phone number only at checkout.

TASKS

1. Add a local mail catcher so verification emails never leave your machine:

```
mailpit:
  image: axllent/mailpit
  ports: ["8025:8025"]
  networks: [idp]
```

```
docker compose up -d mailpit      # inbox UI: http://localhost:8025
```

2. Create a realm named shop. In *Realm settings* → *Email*, set From to noreply@shop.lab, host mailpit, port 1025, and click *Test connection*. In the *Login* tab, turn on User registration, Email as username, Forgot password, and Verify email. Turn on brute force detection.
3. Require terms acceptance. Under *Authentication* → *Required actions*, enable *Terms and Conditions* and turn on *Set as default action*.
4. Set up progressive profiling. Create a client scope named checkout (type Optional). In *Realm settings* → *User profile* → *Create attribute*, add phone. Mark it required, under *Required when* choose *Scopes are requested* → checkout, and let users view and edit it.
5. Create the client shop-web. Make it public (client authentication off) with standard flow, redirect URI `http://localhost:9999/*`, and *Consent required* on. Add checkout under its Optional client scopes.
6. Register as a customer. Open this URL:

```
http://keycloak:8080/realms/shop/protocol/openid-connect/auth?client_id=shop-
web&response_type=code&scope=openid&redirect_uri=http://localhost:9999/cb
```

Click *Register* and use jo@example.com. Accept the terms, open the verification email in Mailpit and click the link, then grant consent. The browser ends on localhost:9999/cb?code=... with “connection refused”. That is expected: nothing is listening on that port, and the code in the URL proves the login worked.

```
Registration fields asked: email, first name, last name, password      ← phone NOT asked yet
Consent screen: "shop-web" wants: Email address, User profile
```

7. Check out. Use the same URL with `scope=openid%20checkout`. Keycloak now asks for **only** the phone number, then asks for consent to the new scope.
8. Review the consent record. Open `http://keycloak:8080/realms/shop/account` as Jo, go to *Applications*, and revoke shop-web. Log in again: consent is requested again.
9. Write `ciam-design.md` as a friction ledger. For every step, give the user cost, the security value, and the alternative you rejected: registration, verification, terms, consent, progressive profile, MFA (optional, offered after the first order), social login (a tradeoff you describe without implementing it), bot defense, and account-takeover signals.

DONE WHEN

- ☐ Jo registers, verifies, accepts the terms, and consents.
- ☐ The phone number is requested only with the checkout scope.
- ☐ Consent revocation works.
- ☐ The friction ledger justifies every step.

Break it

Account enumeration. Use *Forgot password* with `jo@example.com` and then with `nobody@example.com`, and compare the messages. Then try to *Register* with `jo@example.com` again. One flow leaks whether an account exists. Which one? What would you change, and at what cost to real customers?

Reset: delete test customers under *shop* → *Users*. `./snapshot.sh lab26-end`.

LAB 27

Decentralized Identity and the Future of Credentials

Prerequisites: Python 3; pip install cryptography · Artifact: verifiable-credential flow diagram with holder, issuer, and verifier

Objective. Build a working model of a verifiable credential with selective disclosure. An issuer signs, a holder presents only what is needed, and a verifier checks it without ever contacting the issuer. The format is simplified from the SD-JWT approach. It teaches the mechanics and is not an interoperable implementation.

TASKS

1. Save notebook/lab27/vc.py:

```
import json, hashlib, os, base64
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
b64 = lambda b: base64.urlsafe_b64encode(b).decode().rstrip("=")
digest = lambda d: b64(hashlib.sha256(json.dumps(d).encode()).digest())

# ISSUER – Halden HR. Each claim is salted and hashed; only the hashes are signed.
issuer_key = Ed25519PrivateKey.generate(); issuer_pub = issuer_key.public_key()
claims = {"name": "Priya Shah", "employer": "Halden and Co", "title": "Sales Analyst",
          "salary_band": "B3", "start_year": "2024"}
disclosures = {k: [b64(os.urandom(16)), k, v] for k, v in claims.items()}
payload = json.dumps({"iss": "did:example:halden-hr", "type": "EmploymentCredential",
                     "_sd": sorted(digest(d) for d in disclosures.values())}).encode()
wallet = {"payload": payload, "sig": issuer_key.sign(payload), "disclosures": disclosures}

# HOLDER – shows a landlord only employer and title
shown = [wallet["disclosures"][k] for k in ("employer", "title")]

# VERIFIER – needs only the issuer's public key
issuer_pub.verify(wallet["sig"], wallet["payload"])
print("signature: valid, issuer", json.loads(wallet["payload"])["iss"])
sd = json.loads(wallet["payload"])["_sd"]
for d in shown:
    print(f"  {d[1]:9} = {d[2]:14} {'verified' if digest(d) in sd else 'NOT IN CREDENTIAL'}")
print("  undisclosed claims:", len(sd) - len(shown))
```

```
cd notebook/lab27 && python3 vc.py
```

```
signature: valid, issuer did:example:halden-hr
  employer   = Halden and Co   verified
  title      = Sales Analyst   verified
  undisclosed claims: 3
```

2. Answer in vc-notes.md: What did the verifier learn? What can it not learn, such as the salary band? Why is each claim salted before hashing? Try working out a salary band from its hash without the salt.

3. Draw `vc-flow.png` with three roles: issuer (Halden HR), holder (Priya's wallet), and verifier (a landlord). Show issuance, storage, presentation, and verification. Add a dotted line from the verifier to a trust registry that says whether `did:example:halden-hr` is a real employer. Label what each party must trust.
4. Write a realistic assessment: three things that work today, three still emerging (revocation at scale, wallet recovery, whether verifiers will accept credentials), and where Halden could use this alongside Keycloak, for example as proof of employment for partners.

DONE WHEN

- ☐ The script verifies two claims and keeps three hidden.
- ☐ The diagram shows the trust relationships, not just the arrows.
- ☐ The assessment separates what is production-ready from what is aspirational.

Break it

Play a dishonest holder twice. First, change the disclosed title to "Sales Director" before the verifier step. Then, separately, change one character of the signed payload itself, for example the issuer name. Record the two different failures and explain which protection catches each one.

Reset: none; no lab state changed.

Architecture and Operations

Hybrid identity, reference architecture, running the program, systematic troubleshooting, and what comes next.

LAB 28

Hybrid and Multi-Cloud Identity

Prerequisites: Lab 25 snapshot · Artifact: hybrid identity architecture connecting the on-premises directory to the cloud tenant

Objective. Treat the halden realm with its directory as “on-premises” and a new cloud realm as the cloud tenant. Connect them with federated authentication, find where the connection breaks, and audit a cloud permission set for sprawl.

TASKS

1. Register the cloud tenant with on-premises. In halden, create the confidential client `cloud-broker` with redirect URI `http://keycloak:8080/realms/cloud/broker/halden-onprem/endpoint`. Copy its secret.
2. Create the realm `cloud`. In *Identity providers* → *OpenID Connect v1.0*, set alias `halden-onprem`, discovery endpoint `http://keycloak:8080/realms/halden/.well-known/openid-configuration`, client ID `cloud-broker`, and the secret. Set *Sync mode* to *Force*, so attributes update from on-premises at every login.
3. Send every cloud login to on-premises. Go to *Authentication* → *Flows* → *browser*, open the settings of *Identity Provider Redirector*, and set the default identity provider to `halden-onprem`.
4. Test it. Open `http://keycloak:8080/realms/cloud/account` in a private window. You are redirected to the Halden login. Sign in as `sortiz` and you land in the cloud account console. In the admin console, *cloud* → *Users* → *sortiz* → *Identity provider links* shows the link to `halden-onprem`.
5. Find the seam. Keep Sam's cloud session open, and in a terminal run `./jml.sh leaver sortiz`. Reload the cloud account console.

```
on-prem: sortiz moved to ou=disabled, cannot log in
cloud:   session still active; user sortiz still exists, enabled
```

Write down the two fixes: OIDC back-channel logout from halden to cloud, and a deprovisioning push that disables the cloud account.

6. Audit cloud entitlements. Save this sample role set as `notebook/lab28/cloud-roles.json` and search it for wildcards:

```
[{"role": "reporting-reader", "assigned": ["svc-reporting"], "actions": ["storage:Get",
"storage:List"], "resources": ["finance-bucket/*"]},
{"role": "dev-helper", "assigned": ["mruiz"], "actions": ["*"], "resources": ["*"]},
{"role": "ops-deploy", "assigned": ["dokafor"], "actions": ["compute:*", "iam:PassRole"],
"resources": ["*"]}]
```

```
jq -r '.[ ] | select((.actions + .resources) | any(test("\\*$"))) | "\\(.role)\\t\\
(.assigned|join(","))\\t\\(.actions|join(","))"' \
notebook/lab28/cloud-roles.json
```

reporting-reader	svc-reporting	storage:Get,storage:List
dev-helper	mruiz	*
ops-deploy	dokafor	compute:*,iam:PassRole

Right-size each role. Explain why `iam:PassRole` on `*` is an escalation path, and why a wildcard limited to a single bucket is usually acceptable.

7. Draw `hybrid-architecture.png`. Show the directory, the on-premises IdP, the cloud realm, the trust between them, the direction of attribute flow, the deprovisioning path, and the seams you found. Add a paragraph on portability: what it would take to point `ccloud` at a different on-premises IdP.

DONE WHEN

- A cloud login is authenticated on-premises.
- The leaver seam is demonstrated, and its fixes are written down.
- Every wildcard role is right-sized.

Break it

Pivot across the trust. In the `ccloud` realm, create a local user `pshah` with a known password before Priya's first federated login. Then have Priya sign in through `halden-onprem`. What does Keycloak's first-broker-login flow do with the collision? Why is automatic account linking by email dangerous?

Reset: `./restore.sh lab25-end` brings Sam back and removes the `ccloud` realm; or keep the realm and re-run the joiner for Sam. `./snapshot.sh lab28-end`.

LAB 29

IAM Architecture and Design

Prerequisites: all previous deliverables · Artifact: complete IAM reference architecture for Halden

Objective. Bring 28 labs together into one design that you can defend. Then test the design's most important failure case: the IdP going down.

TASKS

- 1. Write the requirements in `notebook/lab29/requirements.md`. Cover populations (workforce, contractors, partners, customers, non-human identities), resources by tier, regulatory drivers (your Lab 20 framework), and constraints such as budget, skills, and existing systems.
- 2. Build the layer map. Make a table with one row per layer: directory, authentication, authorization, lifecycle, governance, privileged access, non-human identity, federation, CIAM. Columns: the lab component that implements it, the production equivalent you would choose, and the lab that proves it works.
- 3. Choose where each model applies. Say where RBAC sets the baseline (Lab 13), where ABAC adds context (Lab 14), and where zero trust decisions sit (Lab 16). Draw the one path a request takes through all three.
- 4. Decide build, buy, or assemble for each capability. For every layer, write the decision and one sentence of rationale. The lab *assembled* open-source parts; say for each whether that would still hold at 4,000 employees.
- 5. Test the IdP outage. With Priya signed into both apps, stop the IdP and observe for two minutes:

```
./snapshot.sh pre-outage; docker compose stop keycloak; date
```

Fill in this table:

	t+0	t+2 min	why
Grafana (existing)	?	?	
intranet (existing)	?	?	(cookie-refresh=1m from Lab 11)
new login, any app	?	?	
break-glass admin path	?	?	

Start Keycloak again with `docker compose start keycloak`.

- 6. Assemble `reference-architecture.md`: a context diagram, the layer map, the models, the integration patterns (OIDC, SAML, header-based proxy, LDAP, API provisioning), the availability design (IdP clustering, directory replication, a vault that doesn't depend on the IdP), the blast-radius notes from Lab 05, and a list of mistakes this design avoids.

DONE WHEN

- Every layer maps to a lab that demonstrated it.
- The outage table is filled in from observation, not guesses.
- The architecture names its single points of failure and how each is mitigated.

Break it

During the outage, try to reach Vault and read the `adm-dokafor` credential. Does your break-glass path depend on anything that was down? Change the design until it doesn't.

Reset: `./restore.sh pre-outage` if anything stays unhealthy. `./snapshot.sh lab29-end`.

LAB 30

Operating an IAM Program

Prerequisites: Labs 18-22 · Artifact: operating model with metrics dictionary and operational calendar

Objective. Measure the program with a script, not a slide. Run a timed incident response for a compromised account, and write the calendar that keeps the program running.

TASKS

1. Save metrics.sh:

```
#!/usr/bin/env bash
set -euo pipefail; source .env; BASE=dc=halden,dc=lab
B=(-x -LLL -H ldap://localhost -D cn=admin,$BASE -w "$LDAP_ADMIN_PASSWORD")
m() { printf '%-32s %s\n' "$1" "$2"; }
m standing_privilege_humans "$(ldapsearch "${B[@]}" -b cn=grp-lab-admins,ou=groups,$BASE
member | grep -c 'ou=people' || true)"
leftover=0; for dn in $(ldapsearch "${B[@]}" -b ou=disabled,$BASE "(uid=*)" dn | sed -n
's/^dn: //p'); do
    leftover=$((leftover + $(ldapsearch "${B[@]}" -b ou=groups,$BASE "(member=$dn)" dn | grep -c
^dn || true))); done
m disabled_accounts_with_access "$leftover"
m nhi_without_owner "$(ldapsearch "${B[@]}" -b ou=service,$BASE '(!((description=Owner:*))' dn
| grep -c ^dn || true)"
f=$(ls notebook/lab19/campaign-*.csv | tail -1); total=$((($wc -l < $f) - 1))
done_=$(awk -F, 'NR>1 && $7!="" $f | wc -l); m review_completion "$((100 * done_ / total))%"
m jit_grants_last_30d "$(grep -c GRANT logs/jit.log || true)"
```

standing_privilege_humans	0	
disabled_accounts_with_access	0	
nhi_without_owner	1	← the placeholder: document it as an
accepted exception		
review_completion	100%	
jit_grants_last_30d	3	

- Write metrics-dictionary.md. For each metric, give its definition, source, target, owner, and the way it could mislead. Add orphaned accounts (Lab 18), time to deprovision (the HR end date compared with the modifyTimestamp on the entry in ou=disabled), and access-request cycle time. List two vanity metrics to avoid, such as “number of accounts provisioned”.
- Run a timed incident. Scenario: *Priya's password and a session cookie were phished*. Start a timer, then contain the account:

```
source kc_token.sh; API=http://keycloak:8080/admin/realms/halden
UID=$(curl -s -H "Authorization: Bearer $KC" "$API/users?username=pshah&exact=true" | jq -r
'[0].id')
curl -s -X POST -H "Authorization: Bearer $KC" "$API/users/$UID_/logout" #
kill IdP sessions
ldappasswd -x -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w "$LDAP_ADMIN_PASSWORD"
uid=pshah,ou=people,dc=halden,dc=lab # scramble
GID=$(curl -s -u admin:$GRAFANA_ADMIN_PASSWORD "localhost:3000/api/users/lookup?
loginOrEmail=pshah" | jq .id)
curl -s -X POST -u admin:$GRAFANA_ADMIN_PASSWORD localhost:3000/api/admin/users/$GID/logout
# kill app sessions
curl -s -H "Authorization: Bearer $KC" "$API/events?user=$UID_&max=50" | jq -r '.[0] | "\
(.time/1000|todate) \(.type) \(.ipAddress) \(.clientId)'"'
```

```
{"message": "User logged out"}
2026-09-24T12:01:13Z LOGIN 172.18.0.1 intranet
2026-09-24T11:40:02Z CODE_TO_TOKEN 172.18.0.1 grafana
...
```

Record the time to containment. Confirm that a stolen _oauth2_proxy cookie stops working within a minute, thanks to Lab 11.

4. Turn the incident into runbook-compromised-credential.md, and draft two more runbooks: a rogue admin, and mass revocation (for example, a partner breach, using Lab 25).
5. Write operating-calendar.md. Daily: sync logs, JIT log review, detection alerts. Weekly: NHI owner check, orphan report. Monthly: metrics review, app onboarding intake. Quarterly: certification campaign and evidence pack. Yearly: maturity assessment, break-glass test, policy review. Add the team roles and a change-communication note for access being removed.

DONE WHEN

- metrics.sh runs, and every metric appears in the dictionary.
- Containment took under 10 minutes and was timed.
- The calendar covers every recurring control from Labs 17–24.

Break it

A misleading metric. Show that review_completion can read 100% while every decision is “keep”. Add a companion metric that exposes rubber-stamping, and give it a target.

Reset: set Priya's password back to \$USER_PASSWORD with ldappasswd -s. ./snapshot.sh lab30-end.

LAB 31

Troubleshooting IAM

Prerequisites: Lab 30 snapshot · Artifact: layered troubleshooting checklist applied to staged failures

Objective. A script breaks the lab in a way you don't know in advance. Using a checklist that works through the stack layer by layer, find the fault, fix it, and record the evidence. Repeat until you have solved all four faults.

TASKS

1. Write the checklist first, in notebook/lab31/checklist.md. Start from this skeleton and add a check and a command for every layer:

```
0. Scope      Who is affected (one user, a group, everyone)? Which apps? Since when?
1. Which A?   Fails before credentials (config/trust), at credentials (authn), after
              login (authz)?
2. Directory  ldapwhoami as the user; ldapsearch their groups; Keycloak "Test connection"
3. IdP        Events → User events for the user; docker compose logs keycloak --since 10m
4. Trust      Client redirect URIs, secrets, issuer, SAML metadata and certificates
5. Claims     Clients → Evaluate: does the token carry the groups/roles the app expects?
6. App        App logs: docker compose logs grafana|oauth2-proxy --since 10m
7. Effective  Lab 15 effective-access script: what can this user actually reach?
```

2. Save stage_failure.sh. It records which fault it chose in an encoded file so you can check your answer afterward.

```
#!/usr/bin/env bash
set -euo pipefail; source .env
KA() { docker compose exec -T keycloak /opt/keycloak/bin/kcadm.sh "$@"; }
KA config credentials --server http://localhost:8080 --realm master --user admin --password
"$KC_ADMIN_PASSWORD" >/dev/null
cid() { KA get clients -r halden -q sc-camel-client-id=$1 --fields id --format csv --noquotes
| tr -d '\r'; }
n=${1:-$((RANDOM % 4 + 1))}; echo $n | base64 > .fault
case $n in
1) KA create clients/$(cid grafana)/client-secret -r halden >/dev/null ;;
2) printf "dn: cn=app-intranet-users,ou=groups,dc=halden,dc=lab\nchangetype: modify\ndelete:
member\nmember: uid=pshah,ou=people,dc=halden,dc=lab\n" \
| ldapmodify -x -H ldap://localhost -D cn=admin,dc=halden,dc=lab -w
"$LDAP_ADMIN_PASSWORD" >/dev/null ;;
3) LC=$(KA get components -r halden -q name=ldap --fields id --format csv --noquotes | tr -
d '\r')
KA update components/$LC -r halden -s 'config.bindCredential=["wrong-password"]' ;;
4) KA update clients/$(cid grafana) -r halden -s 'redirectUris=
["http://localhost:3000/login/oauth"]' ;;
esac
echo "Fault staged. Ticket: \"Priya says something is broken.\""
```

```
./snapshot.sh pre-fault; chmod +x stage_failure.sh; ./stage_failure.sh
```

```
Fault staged. Ticket: "Priya says something is broken."
```

3. Diagnose. Work through the checklist in order and write down every command and what it showed. Don't guess, and don't restore the snapshot until you can name the fault, the layer it sits in, and the evidence that proves it.
4. Fix it with the smallest possible change (not by restoring the snapshot). Confirm Priya can reach both apps, then check your answer with `base64 -d .fault`.
5. Run `./restore.sh pre-fault`, then stage the next fault with `./stage_failure.sh 1, 2, 3, 4` until you have solved all four. Keep a record for each: the symptom, the first misleading clue, the layer, the root cause, and the fix.

DONE WHEN

- ☐ All four faults are diagnosed, each with evidence from a specific layer.
- ☐ Each fix is the smallest possible change, not a restore.
- ☐ You have improved the checklist with at least one step you wished you had had.

Break it

Write fault 5 yourself and trade scripts with a classmate. Ideas: expire a SAML signing key (Lab 25), delete the groups mapper from the grafana client (Lab 12), or stop OPA while an app depends on it (Lab 16).

Reset: `./restore.sh pre-fault; delete .fault`.

LAB 32

Building Skills and Where to Go Next

Prerequisites: all labs · Artifact: skills self-assessment and personal development plan

Objective. Run one final check of the whole lab, rate yourself honestly against every part of the course, and leave with a 90-day plan and a lab that keeps growing.

TASKS

1. Run the capstone check. Save `verify_lab.sh`:

```
#!/usr/bin/env bash
source .env; BASE=dc=halden,dc=lab; A=(-x -LLL -H ldap://localhost -D cn=admin,$BASE -
w "$LDAP_ADMIN_PASSWORD")
check() { if eval "$2" >/dev/null 2>&1; then echo "PASS $1"; else echo "FAIL $1"; fi; }
check "directory: read-only bind works" 'ldapwhoami -x -H ldap://localhost -D uid=svc-
keycloak,ou=service,$BASE -w "$SVC_KC_PW"'
check "IdP: discovery published" 'curl -sf http://keycloak:8080/realms/halden/.well-
known/openid-configuration'
check "leaver: ltran not in ou=people" '! ldapsearch "${A[@]}" -b ou=people,$BASE "
(uid=ltran)" dn | grep -q ^dn'
check "PAM: no standing human admins" '! ldapsearch "${A[@]}" -b cn=grp-lab-
admins,ou=groups,$BASE member | grep -q ou=people'
check "policy engine healthy" 'curl -sf localhost:8181/health'
check "NHI: workload token issued" 'curl -sf
http://keycloak:8080/realms/halden/protocol/openid-connect/token -d
grant_type=client_credentials -d client_id=svc-reporting -d
client_secret="$SVC_REPORTING_SECRET"'
check "governance: evidence pack builds" './evidence.sh'
check "segmentation: apps cannot see LDAP" '! docker run --rm --network halden_apps
nicolaka/netshoot nc -zw2 openldap 389'
```

```
PASS directory: read-only bind works
PASS IdP: discovery published
PASS leaver: ltran not in ou=people
PASS PAM: no standing human admins
PASS policy engine healthy
PASS NHI: workload token issued
PASS governance: evidence pack builds
PASS segmentation: apps cannot see LDAP
```

Fix anything that fails by going back to the lab that built it.

2. Rate yourself in `self-assessment.md`, from 1 (I recognize it) to 4 (I could teach it and troubleshoot it), for each part: foundations, authentication, authorization, governance, privileged and non-human identity, federation and CIAM, architecture and operations. For every score below 3, cite the lab you would redo.

3. Pick a specialization (IGA, PAM, CIAM, federation, non-human identity, or identity security) and write down why, based on which labs you enjoyed and which scores were highest.
4. Write `90-day-plan.md`. Days 1–30: redo your two weakest labs and extend the lab (ideas: add a replica directory, put TLS on Keycloak, and move X.509 authentication from Lab 10 into the IdP). Days 31–60: a project in your specialization, such as a SCIM connector, an OPA sidecar for Grafana, or a service-account discovery tool. Days 61–90: publish a write-up, and choose a certification or vendor track whose syllabus you have checked against your gaps.
5. Do a final revision of the Lab 01 concept map. Compare it with your first version and write one paragraph on what changed.

DONE WHEN

- ☐ `verify_lab.sh` is all PASS.
- ☐ The self-assessment cites specific labs as evidence.
- ☐ The 90-day plan has one concrete, demonstrable output per month.

Reset: `./snapshot.sh course-complete`. Keep this lab running; it is your practice environment from here on.

Answer Key

Solutions to break-it exercises and key design questions. Attempt each lab before reading its entry.

Lab 01. Wrong password: authentication. Payroll folder visible: authorization is where it shows, but the cause is administration, because someone granted the access. Nobody knows who approved: audit (and administration, since approval is part of administering access). Wrong manager: administration. Statements that seem to fit two boxes usually describe a symptom in one A and a cause in another. Lab 31 relies on telling the two apart.

Lab 02. 1005 Paul Shah produces the same key as Priya (pshah) and replaces her in the hr dictionary, so her mailboxes and CRM login are attributed to Paul. The only data every source shares is the name, so improve the key (full first name + surname, then check the department), and send ambiguous matches to a person. The lasting fix is to write the employee ID into every downstream system at provisioning. Names change with marriage, and email addresses change or get reassigned. An employee ID or an immutable GUID does neither.

Lab 03. Keycloak shows *Invalid parameter: redirect_uri* on its own page before asking for credentials. An IdP only sends authorization codes to registered URIs. That is what stops an attacker from steering a victim's code to a site they control.

Lab 04. With OpenLDAP stopped, pshah fails because Keycloak must bind to LDAP to check her password. The master-realm admin works because it is a local Keycloak user. The directory is a single point of failure for every federated login, which is what replication (multi-provider OpenLDAP, several domain controllers) exists to solve. Error 49 means invalid credentials, and error 32 means the search base doesn't exist. A missing attribute returns the entry with no error, so applications must handle absent attributes explicitly.

Lab 05. Typical top five: Keycloak binds as the directory admin (fixed in Lab 24); a leaver can still log in (Lab 17); admins have no MFA (Lab 08); admin rights are standing (Lab 22); the network is flat and apps can reach LDAP (Lab 16). If anonymous search returned entries, add it as a sixth: it hands attackers a list of usernames to spray.

Lab 06. Both attempts log LOGIN_ERROR, with different error details (*invalid_user_credentials* versus *user_not_found*), but the user sees the same “Invalid username or password” both times. The difference is kept for the audit log and hidden from the user, so the login page can't be used to find out which usernames exist. The code-for-token exchange happens server to server, so tokens never appear in browser history or extensions, and the client secret never leaves the server.

Lab 07. On one CPU core, MD5 allows millions of guesses per second and bcrypt at cost 12 allows about five. GPUs multiply the fast hashes by orders of magnitude. The spray against LDAP locks no one out, because the lab directory has no password-policy overlay, and Keycloak never sees the attempts, because they bypass it. Direct access to the directory is a way around every IdP control. The fix is segmentation (Lab 16), a lockout policy in the directory, and monitoring of failed binds.

Lab 08. The setting is *Look around window* in the OTP policy (default 1, meaning one 30-second step either side). Raising it tolerates more clock drift but keeps each code valid for longer, which widens the window for replaying or guessing a code.

Lab 09. A passkey is bound to the relying-party ID, which is the site's domain. On `127.0.0.1`, the browser won't offer a credential registered for keycloak, however identical the page looks. The browser checks the origin, not the user, so a convincing phishing page gets nothing.

Lab 10. The rogue CA causes a verification failure (*unable to get local issuer certificate*), and the expired certificate causes *certificate has expired*. Sensible thresholds are about 70 for unlocking a phone (low risk, and the user has other factors) and 80 or more for a payment, where a false accept costs more than a retry.

Lab 11. The intranet doesn't notice. After login it relies only on its own cookie and never uses the access token again, so the IdP's token lifetime has no effect. Revocation reaches an app only if the app re-checks: by refreshing tokens, introspecting them, or receiving a back-channel logout.

Lab 12. `whoami` answers because the gateway is the only check in place, and anything already inside the network can go around it. Add enforcement in or next to the app: validate a token or a signed header, require mTLS between the proxy and the app, and use network policy so only the proxy can reach it (Lab 16). This is defense in depth.

Lab 13. Grafana re-evaluates `role_attribute_path` at every login and overwrites local role changes, so the IdP stays the source of truth and local grants can't persist without anyone noticing. Role explosion: $4 \times 3 \times 3 = 36$ roles for one app, which is the moment to move context into attributes (Lab 14). Mining exceptions: `cdiaz2` breaks SoD; `dokafor` mixes business rights with admin rights.

Lab 14. With device missing, `input.device.managed` is undefined, so `not of it is true`, the deny rule fires, and the request is refused: the policy fails closed. With `== false`, the comparison is undefined, the rule never fires, and the request is allowed without any device information (keep `acr` at "2" to see this): the policy fails open. Write deny conditions so that missing data triggers them.

Lab 15. A `max_duration` or `expiry` on requestable access would have removed the "temporary" Sales membership. `grp-sales` isn't in the catalog at all, and access that has never been listed can't be governed.

Lab 16. With `risk` missing, `input.risk < 70` is undefined, `baseline_ok` fails, and the default block applies. That is the right result. With OPA down, the call fails. PEPs for high-tier apps should fail closed. A low-risk app might briefly accept a cached recent decision, but must raise an alert. Decide this per app and write it down.

Lab 17. Error: *Object class violation (65)*, because `groupOfNames` requires at least one member. Fixes: keep a placeholder member in every group (used from Lab 22), or delete the group when its last real member leaves. Also stop the leaver process from aborting partway: log each failed step and carry on disabling access.

Lab 18. The reconciliation pass finds ghost plus the 35 generated users, whose employee numbers 1100–1134 aren't in the HR file: 36 orphans. Don't mass-delete them. Confirm each with a manager, add the real employees to HR, then disable the rest after a notice period. Minimal check: list every `employeeNumber` in `ou=people`, subtract the HR employee IDs, and print what remains.

Lab 19. Before writing the reviewer, add: `if reviewer == uid: reviewer = manager_uid_of(uid)`, where the manager comes from the `manager` DN. Escalate if the manager is also missing. Self-review is a separation-of-duties failure in the review process itself.

Lab 20. `sha256sum -c` prints `q1-access.ldif: FAILED`. But an administrator who can edit the evidence can also regenerate the manifest. Send the manifest (or the whole pack) to storage the IAM team can't modify, such as write-once object storage or a ticket owned by compliance, and timestamp it there.

Lab 21. With every value set to 1, the STALE flags disappear: `vrac6` falls to 45 and `mrui3` to 30, and dormant access is missed. Independent signals: business actions in the app's own logs (payments actually approved) rather than login events, and patterns in login timing, such as the same second every night.

Lab 22. The dev vault keeps everything in memory. After a restart, `adm-dokafor`'s password is unknown to anyone. Recover through break-glass: reset the password as `cn=admin`, then re-onboard the static role. Lessons: a production vault needs high availability and tested backups, and the break-glass path must not depend on the vault.

Lab 23. After the change, Marco's path is `mrui3 → app-grafana-editor → grafana → svc-reporting → cn=admin → openldap → keycloak`, 6 hops. Cutting one edge sent him the long way round. Fix the root causes (a service account binding as directory admin, and secrets stored in apps), and recompute the graph every time access changes.

Lab 24. For each entry in `ou=service`, extract the uid after `Owner:` in `description`, then check that `uid=<owner>,ou=people` exists. Report any that don't. Run it weekly and reassign ownership within five working days, or disable the identity.

Lab 25. With *Use metadata descriptor URL* on, Halden fetches the partner's new key and login keeps working (at most you need *Reload keys*). With it off, the assertion fails signature validation, and an identity-provider error event appears under *Events*. Broken metadata exchange after a key rotation is one of the most common federation outages, and automating metadata refresh prevents it.

Lab 26. *Forgot password* gives the same message for any address, so it doesn't leak. *Register* says the email already exists, so it does. Fix: always show “check your email”, and send an existing customer a “you already have an account” message instead. The cost is a slower, more confusing experience for real customers who have forgotten they registered.

Lab 27. A changed title has a new hash that isn't in the signed `_sd` list, so the output says *NOT IN CREDENTIAL*: the disclosure hashes protect each claim. A changed payload makes `verify` raise `InvalidSignature`: the issuer's signature protects the whole credential. The salt stops a verifier from hashing guesses such as “B1”...“B5” to find the hidden salary band.

Lab 28. Keycloak detects an existing `pshah` and asks the user to link accounts by re-authenticating as the existing local user. It does not merge them silently. Automatic linking by email would hand the pre-created account, and anything it owns, to whoever controls that email address at the other IdP. That is a well-known pivot across trust boundaries.

Lab 29. Typical results: new logins fail everywhere at once. Grafana's existing session keeps working until it needs the IdP. The intranet signs users out within about a minute, because its refresh fails. If Vault or the break-glass credential sits behind SSO, the break-glass path is down too. Keep one offline or independently authenticated route to tier 0.

Lab 30. Completion measures only whether reviewers responded. Pair it with a *revocation rate* (target: not zero, and trending toward low but stable) and the median time spent per decision. A reviewer with 100% “keep” at two seconds per item is rubber-stamping.

Lab 31. Fault 1: Grafana fails after a successful Keycloak login, with a `CODE_TO_TOKEN_ERROR` event and invalid client credentials. Cause: the client secret changed, a trust problem. Update `.env` and recreate Grafana. Fault 2: only the intranet fails, with a 403 for Priya, while Grafana works. The token's `groups claim` lacks `app-intranet-users`, an authorization problem. Fault 3: every LDAP user sees “Invalid username or password” but the master admin works, and *Test authentication* fails. Cause: the directory bind credential. Fault 4: Keycloak shows *Invalid parameter: redirect_uri* before asking for credentials. Cause: client configuration.

Lab 32. No single answer. A good plan names outputs someone else can check: a repository, a demo, a write-up.